

AWS Identity and Access Management Project Report

Overview

We're going to learn about AWS Identity and Access Management (IAM) which helps control who can access what in Amazon Web Services. We'll cover things like users, roles, policies, and groups, and we'll also show you how to actually set them up to keep your AWS resources safe.

But before we get into all that, let's make sure you understand the basics of cloud computing. If terms like "Cloud" sound new to you, it's a good idea to go back and review some earlier materials to get a solid grasp of what it's all about.

Project Goals

- Understand AWS Identity and Access Management (IAM) principles and components.
- Learn to create and manage IAM policies for regulating access to AWS resources securely.
- Apply IAM concepts practically to control access within AWS environments.
- Explore best practices for IAM implementation and security in AWS.

Learning Outcomes

- Recognize IAM components like users, roles, policies, and groups.
- Create and manage IAM policies to define permissions for users and roles.
- Set up IAM users, groups, and roles to control access to AWS services.
- Understand IAM best practices for maintaining security and managing access to AWS resources.

Key Concepts

What is IAM?

IAM, or Identity and Access Management, is the gatekeeper for your AWS resources, deciding who gets in and what they're allowed to do once inside.

Imagine you have a big digital "house" full of all your AWS stuff—your data, your applications, the whole shebang. You don't want just anyone wandering in and messing around with your things, right? That's where IAM steps in.

It's like having your own VIP list for your digital world. IAM helps keep your AWS resources safe, ensuring only the right people get in and can only do what you allow. It's all about keeping your digital house in order and protecting your precious stuff from unwanted guests.

Note: AWS resources are the various services and tools provided by Amazon Web Services (AWS) that users can utilize to build and manage their applications and infrastructure in the cloud.

What is IAM User?

IAM users are individual accounts for different people or entities within your AWS environment.

For example, if you have a team working on a project, you can create separate IAM users for each team member. Each IAM user has a unique username and password, allowing them to access the AWS resources needed for their work.

IAM users help manage and control access to your AWS resources securely, ensuring each user only has access to the resources required for their tasks.

What is IAM Role?

An IAM role defines what someone or something (like an application or service) can do within your AWS account. Each role has a set of permissions that determine which actions it can perform and which AWS resources it can access.

For example, you might have an "admin" role with full access to all resources or a "developer" role that only allows access to certain services for building applications.

Imagine a visitor needing temporary access to your house to fix something. Instead of giving them a permanent key (IAM user), you give them a temporary key (IAM role) that works for a limited time and grants access to specific rooms (AWS resources).

IAM roles are flexible and can be assumed by users, services, or applications as needed. They are used for tasks like granting permissions to AWS services, allowing cross-account access, or providing temporary access to external users. IAM roles enhance security and efficiency by providing controlled access without permanent credentials.

What is IAM Policy?

An IAM policy is a set of rules defining what actions a role or user can take. Think of it as a rulebook outlining which actions are allowed and which are not, ensuring secure and controlled access to your AWS resources.

For example, the rulebook might say the "admin" key (IAM role or user) can open any door and perform any action within the house (AWS resources), while the "viewer" key can only open certain doors and look around without making changes.

IAM policies define permissions for IAM roles and users, specifying which AWS resources they can access and what actions they can take. They are essential for maintaining security and ensuring only authorized actions are performed.

What is IAM Group?

IAM Groups are collections of IAM users. Instead of managing permissions for each user individually, you can organize users into groups based on roles or responsibilities.

Think of IAM Groups as teams based on tasks. You might have a group for developers, another for administrators, and so on. Instead of setting permissions for each person one by one, you set them for the whole group at once.

For example, for a development team, you can create an IAM Group called "Developers" and add all developers to it. Assign permissions to the group, and all developers get the same resource access, simplifying management.

Best Practices

- **Give only the permissions needed:** Avoid granting more access than necessary.
- **Use roles instead of users:** Roles are safer and can be used when needed.
- **Review roles regularly:** Remove unused roles to keep things tidy and secure.
- **Add extra security with MFA:** Use Multi-Factor Authentication for extra protection.
- **Use ready-made policies:** They're safer and easier to use.
- **Keep policies simple:** Create separate policies for different tasks.
- **Keep track of changes:** Record who changes what.
- **Test policies before using them:** Ensure they work as intended before applying.
- **Use descriptive names:** Choose clear names for IAM groups to aid understanding and management.
- **Enforce strong password policies:** Encourage strong passwords with expiration and complexity requirements.

Note (difference between users and roles): Users are like individuals with permanent keys to access resources, tied to specific people. Roles are like special keys providing temporary access, usable by different users or programs as needed. Users are fixed to individuals, while roles are flexible for specific tasks.

For MFA, see Multi-Factor Authentication (MFA) for IAM.

Note on AWS policies:

- **Managed Policies:** Created by AWS, widely used.
- **Customer Managed Policies:** You create and manage them.
- **Inline Policies:** Made for one specific thing.

For further details, refer to [Policies and permissions in IAM](#) in the IAM documentation.

Practical Implementation

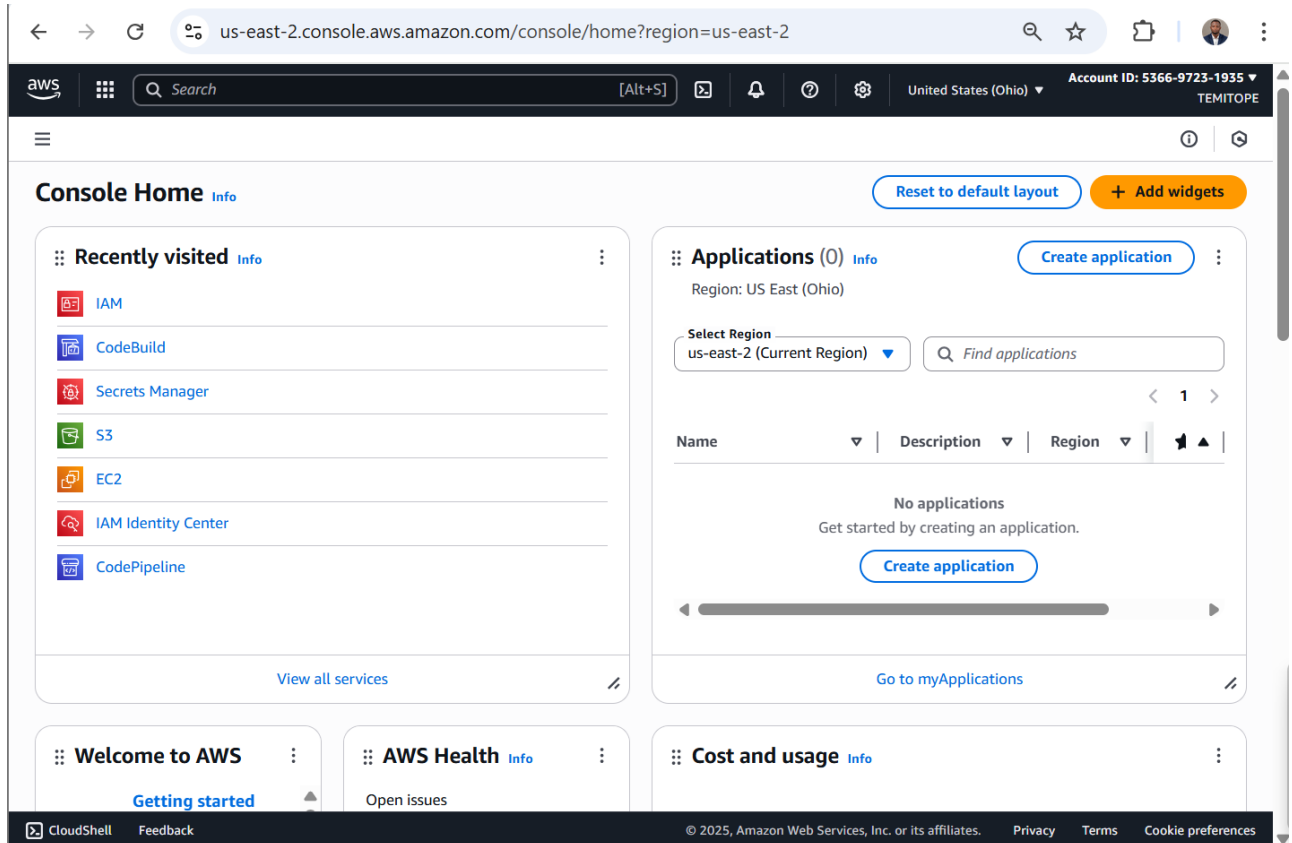
A growth marketing consultancy, GatoGrowFast.com, wants to grant access to AWS resources for employees Seun, Jack, and Ade. The project is divided into three parts:

1. Create a policy granting full EC2 access and assign it to user Seun, along with MFA setup.
2. Create a group, add users Jack and Ade, and create a policy granting full EC2 and S3 access for the group.
3. Enable MFA for user Seun to enhance security.

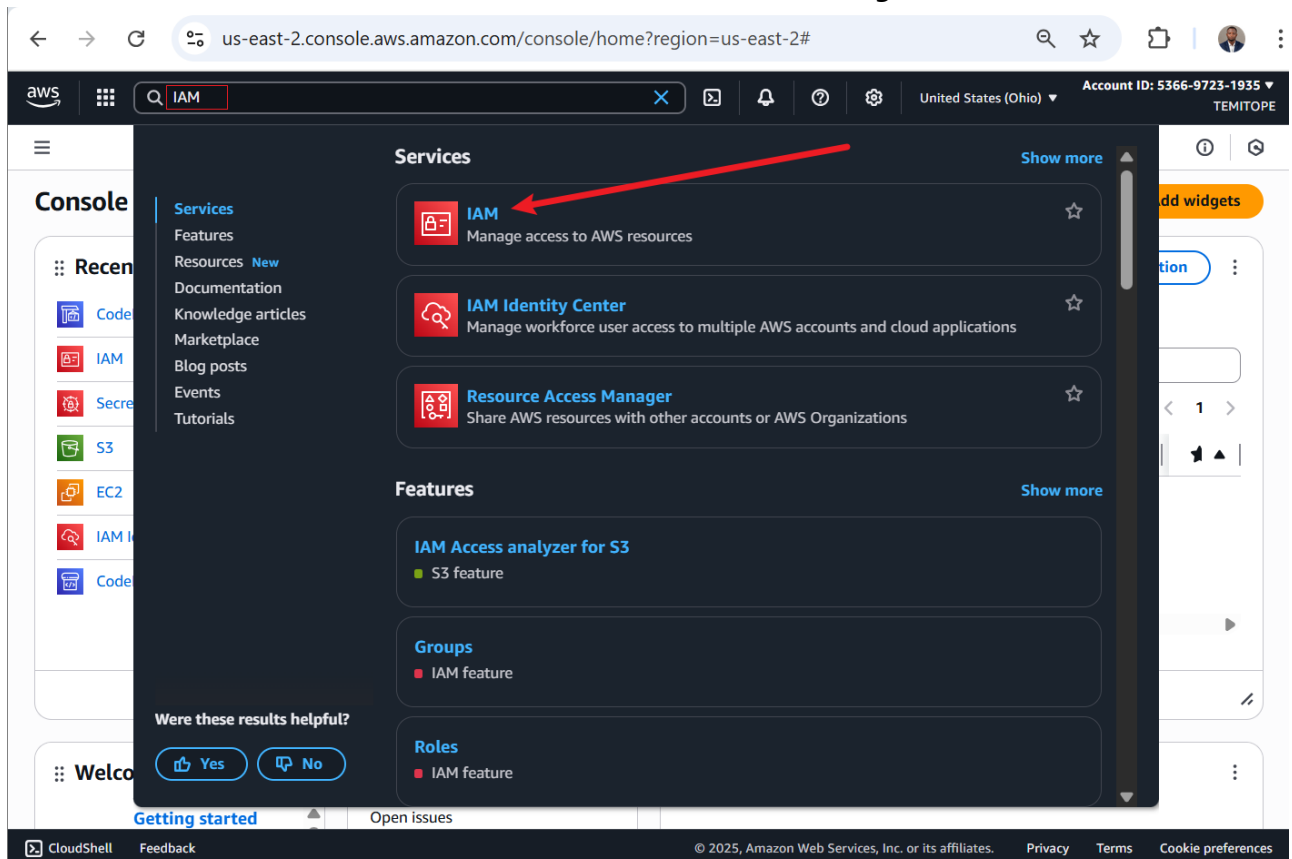
The following steps are ordered numerically based on the provided screenshot names, with each step linked to its corresponding screenshot. Note: Screenshots use "Seun" as the user name, consistent with the provided naming.

Part 1: Create EC2 Policy and IAM User (Seun)

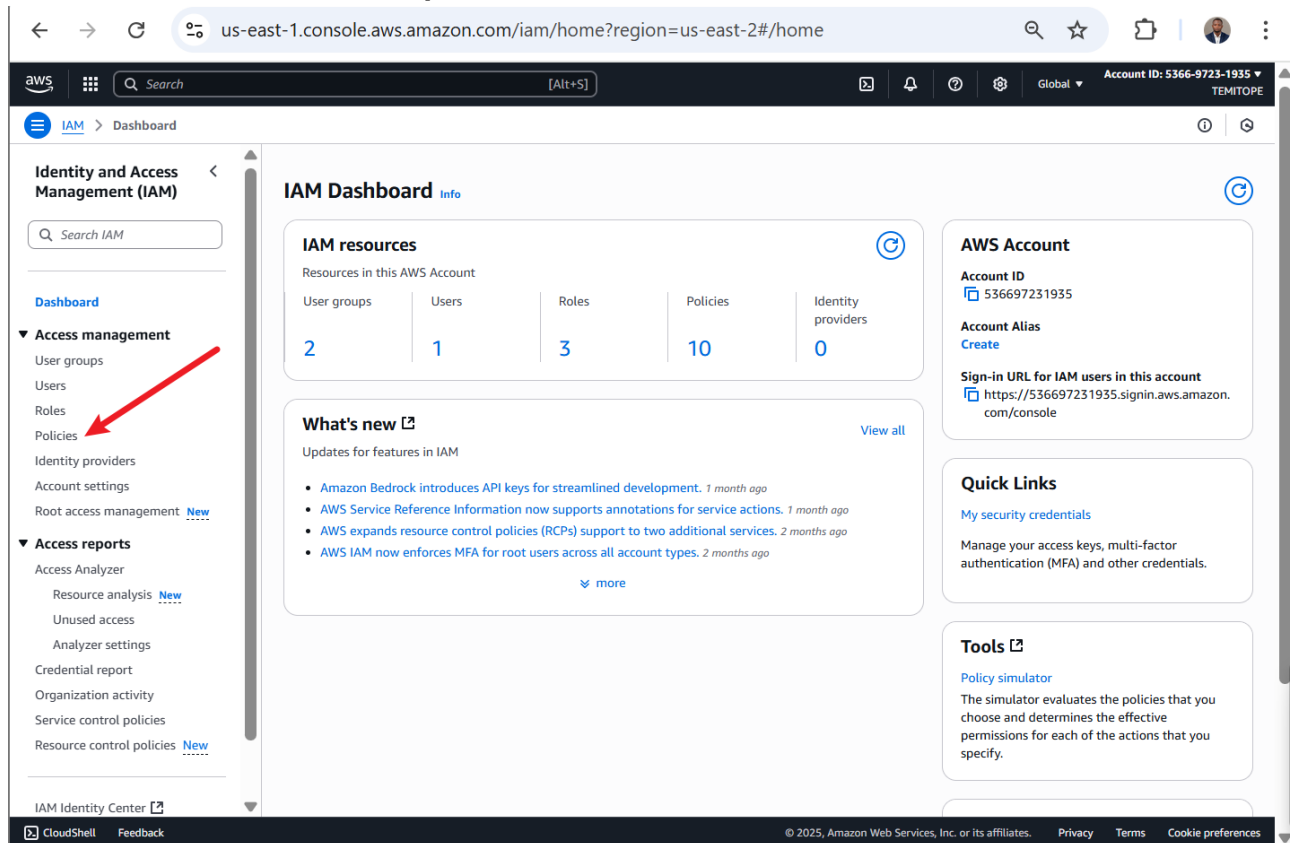
1. Log In and Navigate to Your AWS Console



2. Use the Services Search to Search for IAM and Click to Visit the Page

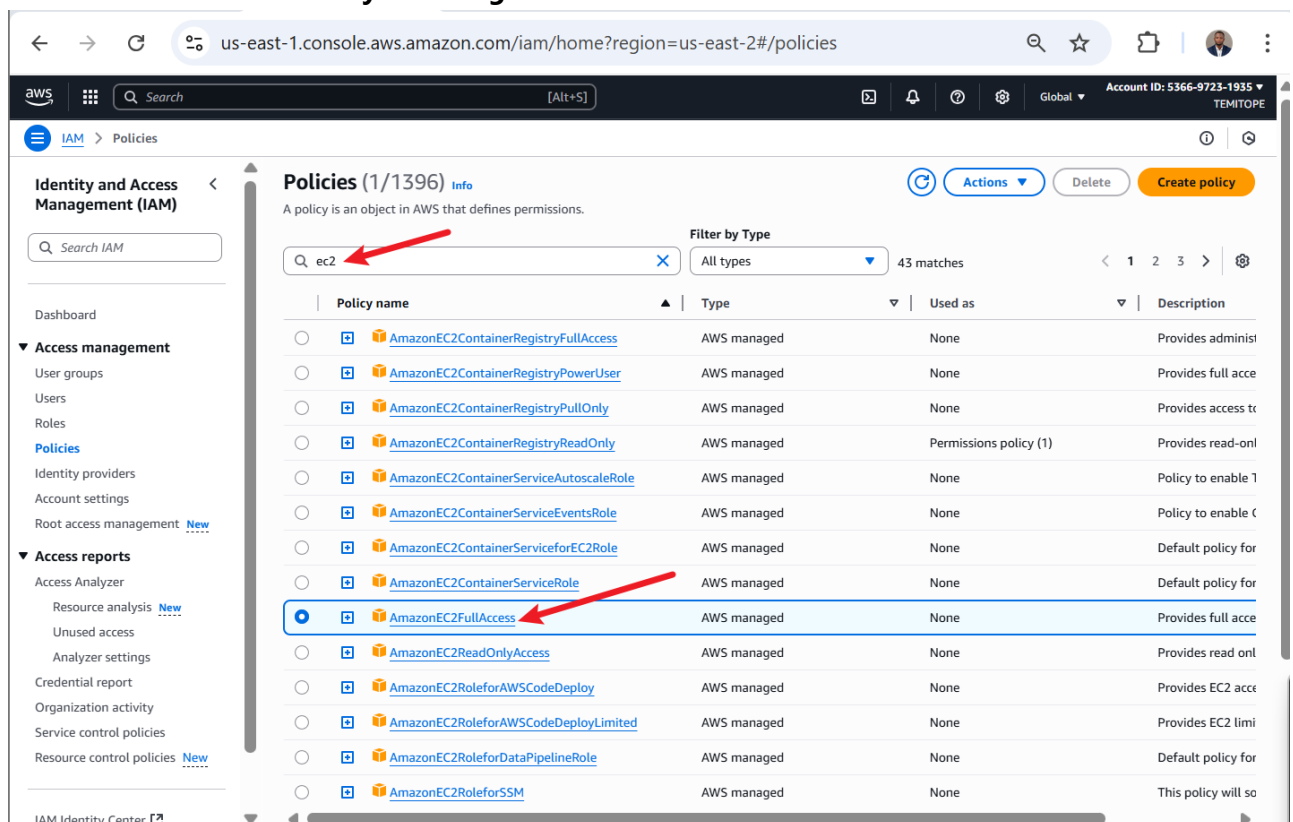


3. Click on Policies on the Menu Option



The screenshot shows the AWS IAM Dashboard in the us-east-1 region. The left-hand navigation menu is expanded, and a red arrow points to the 'Policies' link under the 'Access management' section. The main content area displays the 'IAM Dashboard' with a summary of IAM resources (User groups: 2, Users: 1, Roles: 3, Policies: 10, Identity providers: 0). It also includes a 'What's new' section with updates for IAM features and a 'Tools' section with a 'Policy simulator' link.

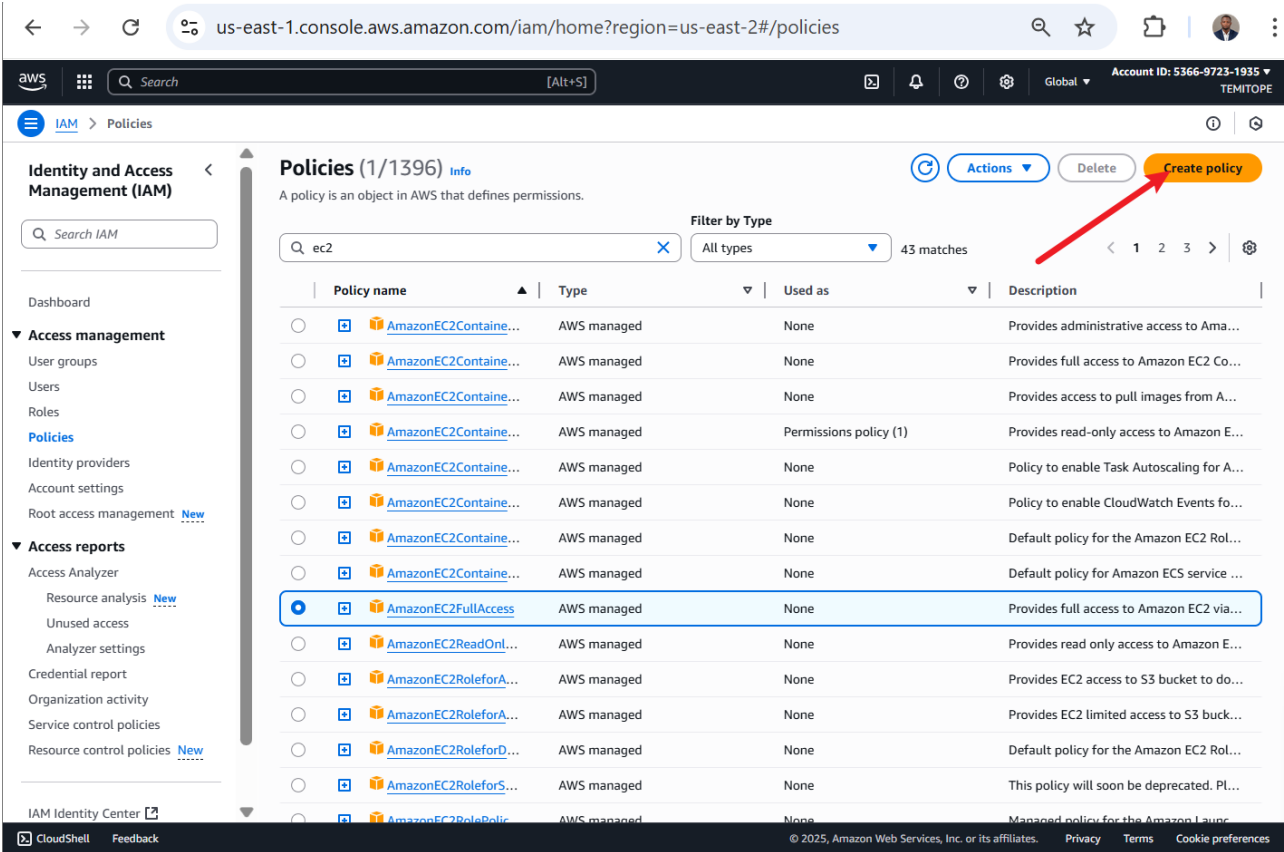
4. Search for EC2 and Grant by Selecting AmazonEC2FullAccess



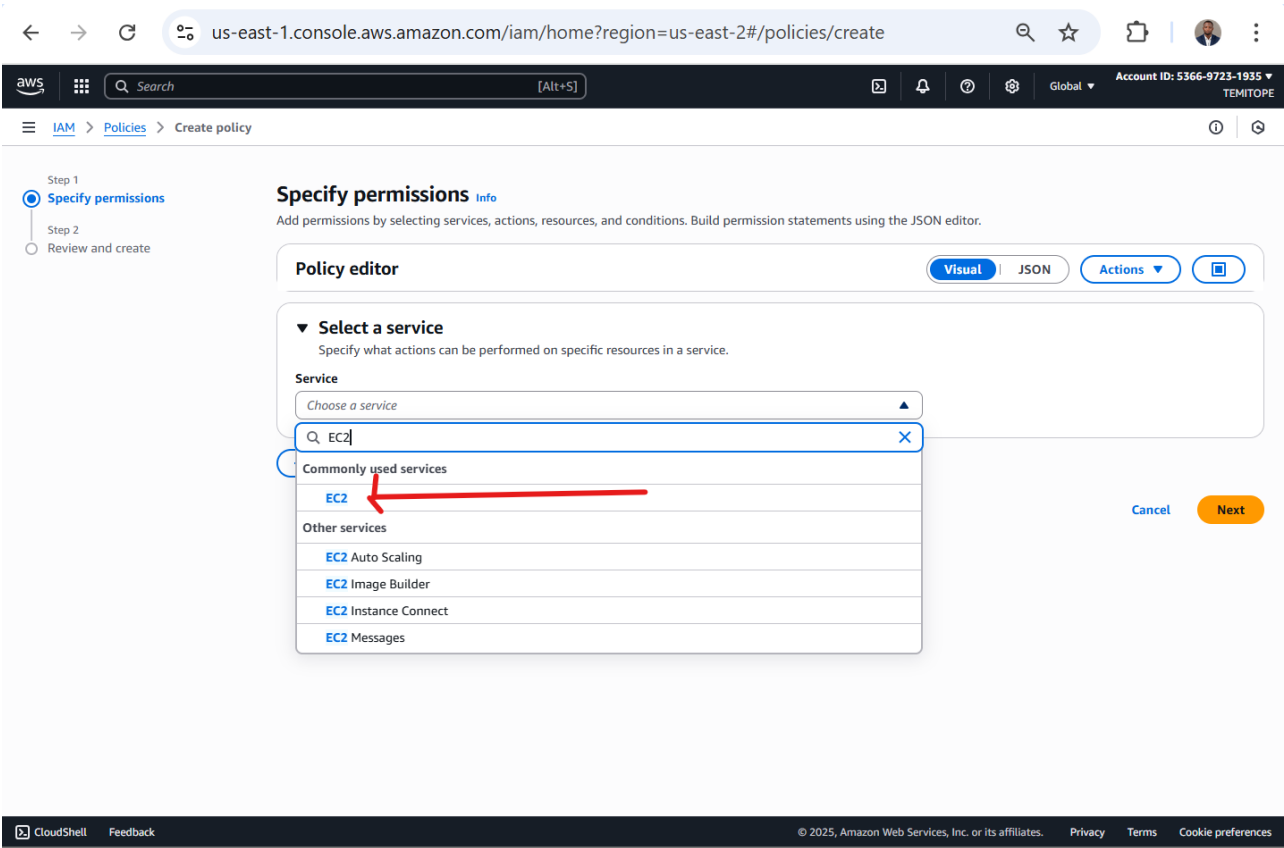
The screenshot shows the AWS IAM Policies page. A search for 'ec2' is performed in the search bar, resulting in 43 matches. The 'AmazonEC2FullAccess' policy is highlighted with a red arrow. The table below lists the policies found:

Policy name	Type	Used as	Description
AmazonEC2ContainerRegistryFullAccess	AWS managed	None	Provides administ
AmazonEC2ContainerRegistryPowerUser	AWS managed	None	Provides full acce
AmazonEC2ContainerRegistryPullOnly	AWS managed	None	Provides access to
AmazonEC2ContainerRegistryReadOnly	AWS managed	Permissions policy (1)	Provides read-onl
AmazonEC2ContainerServiceAutoscaleRole	AWS managed	None	Policy to enable 1
AmazonEC2ContainerServiceEventsRole	AWS managed	None	Policy to enable C
AmazonEC2ContainerServiceforEC2Role	AWS managed	None	Default policy for
AmazonEC2ContainerServiceRole	AWS managed	None	Default policy for
AmazonEC2FullAccess	AWS managed	None	Provides full acce
AmazonEC2ReadOnlyAccess	AWS managed	None	Provides read onl
AmazonEC2RoleforAWSCodeDeploy	AWS managed	None	Provides EC2 acce
AmazonEC2RoleforAWSCodeDeployLimited	AWS managed	None	Provides EC2 limi
AmazonEC2RoleforDataPipelineRole	AWS managed	None	Default policy for
AmazonEC2RoleforSSM	AWS managed	None	This policy will so

5. Click Create Policy



6. Select EC2 Service



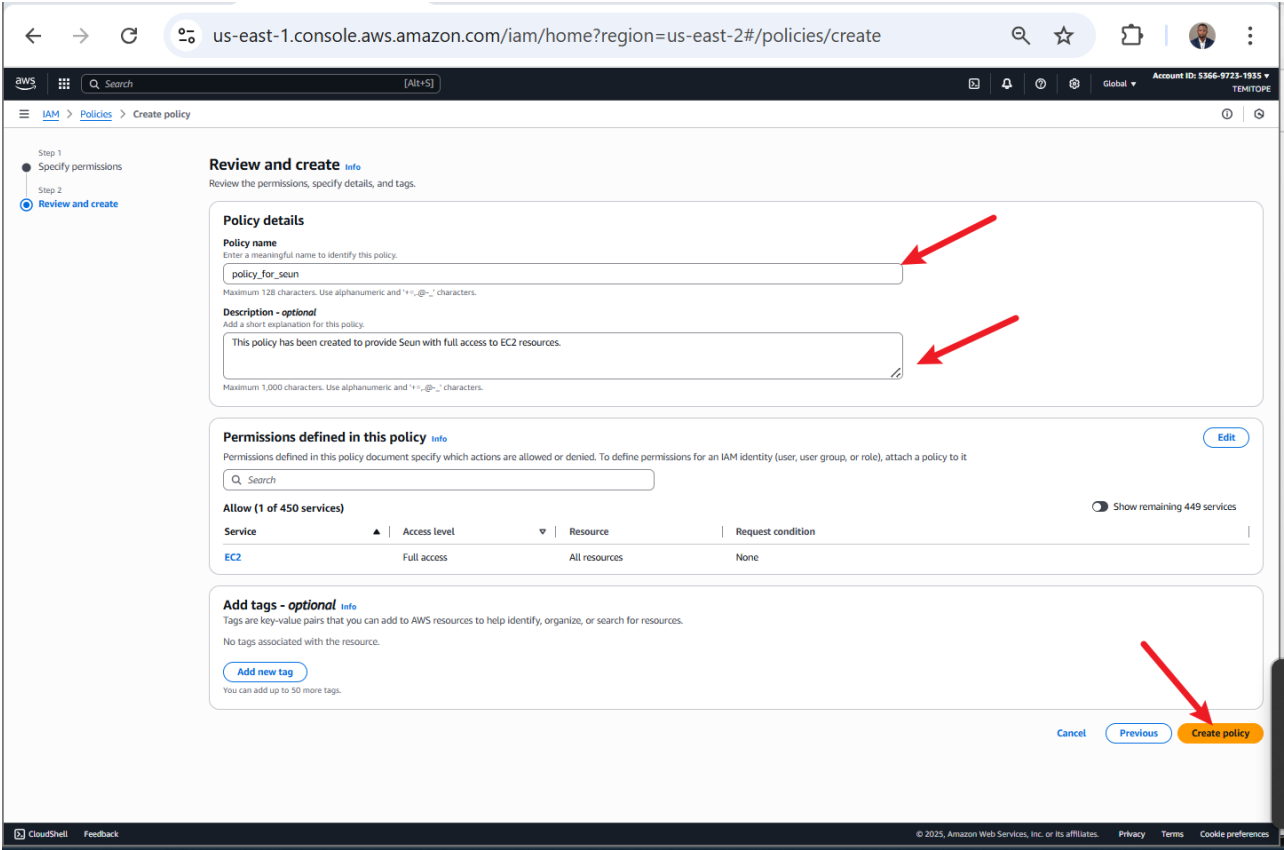
7. Select All EC2 Actions Under Action and Scroll Down

The screenshot shows the AWS IAM console 'Specify permissions' step. The breadcrumb navigation is 'IAM > Policies > Create policy'. The page title is 'Specify permissions' with an 'Info' link. Below the title, it says 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' The 'Policy editor' section has tabs for 'Visual', 'JSON', and 'Actions'. The 'Actions' tab is selected. Under the 'EC2' service, the 'All actions' checkbox is checked. A red arrow points to the 'All EC2 actions (ec2:*)' checkbox under 'Manual actions | Add actions'. The 'Access level' section shows a list of actions: 'List (Selected 194/194)', 'Read (Selected 44/44)', 'Write (Selected 448/448)', 'Permissions management (Selected 16/16)', and 'Tagging (Selected 2/2)'. A warning box states 'Dependent permissions not selected. To grant permissions for the selected resource actions, including additional dependent actions might be required.' The 'Effect' section has 'Allow' selected and 'Deny' unselected. The 'Expand all | Collapse all' link is visible.

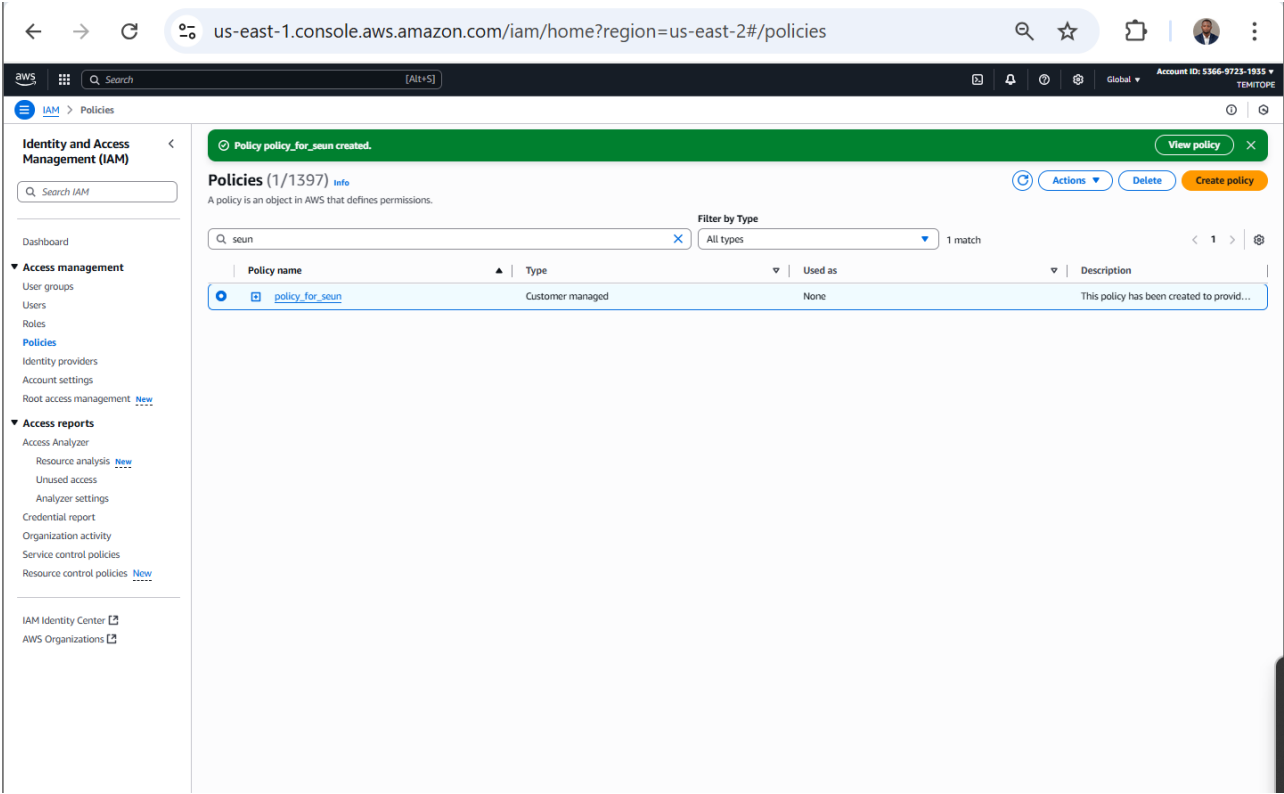
8. Under Resources, Tick All and Click Next Button

The screenshot shows the AWS IAM console 'Specify permissions' step. The breadcrumb navigation is 'IAM > Policies > Create policy'. The page title is 'Specify permissions' with an 'Info' link. Below the title, it says 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' The 'Policy editor' section has tabs for 'Visual', 'JSON', and 'Actions'. The 'Actions' tab is selected. Under the 'EC2' service, the 'All actions' checkbox is checked. A red arrow points to the 'All' radio button under 'Resources'. The 'Request conditions - optional' section is visible. A red arrow points to the 'Next' button. The 'Cancel' button is also visible.

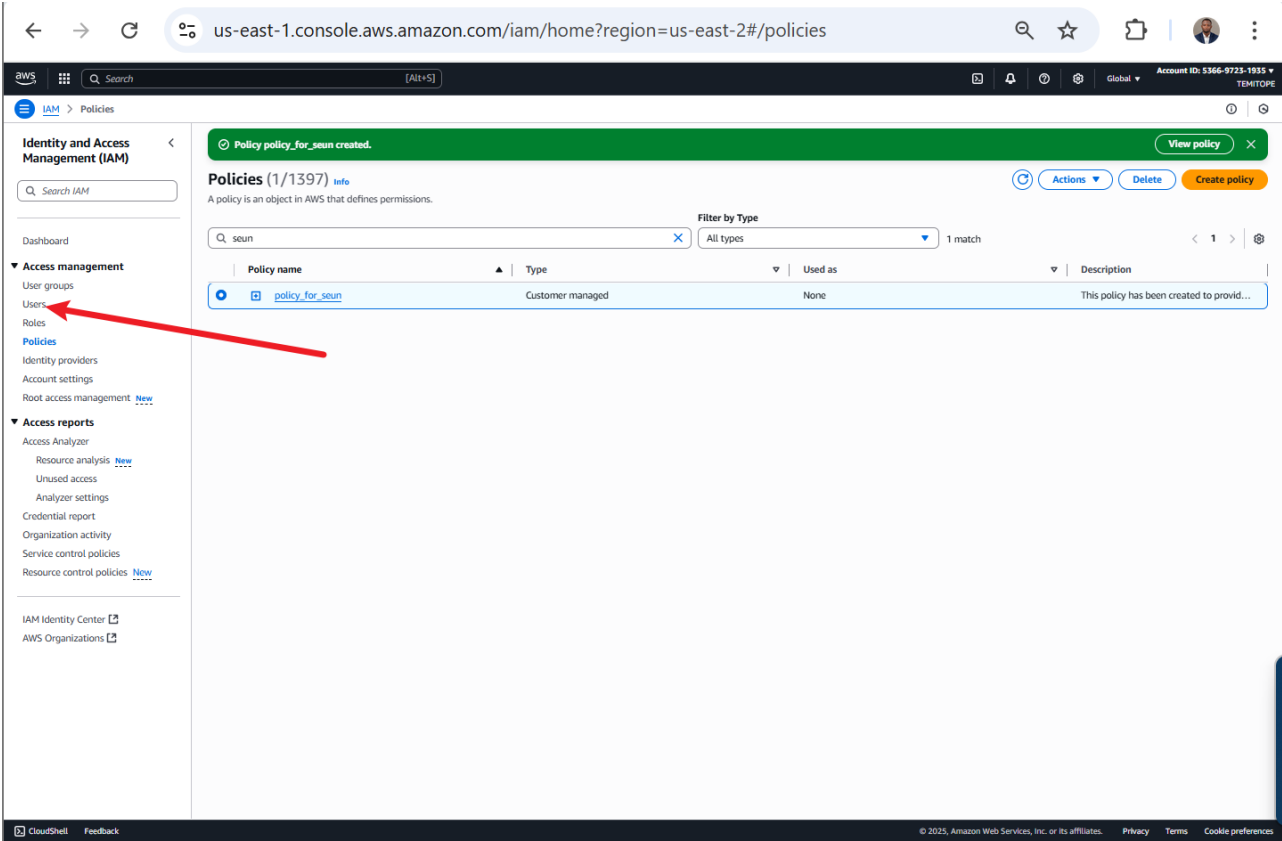
9. Add Policy Names and Create Policy



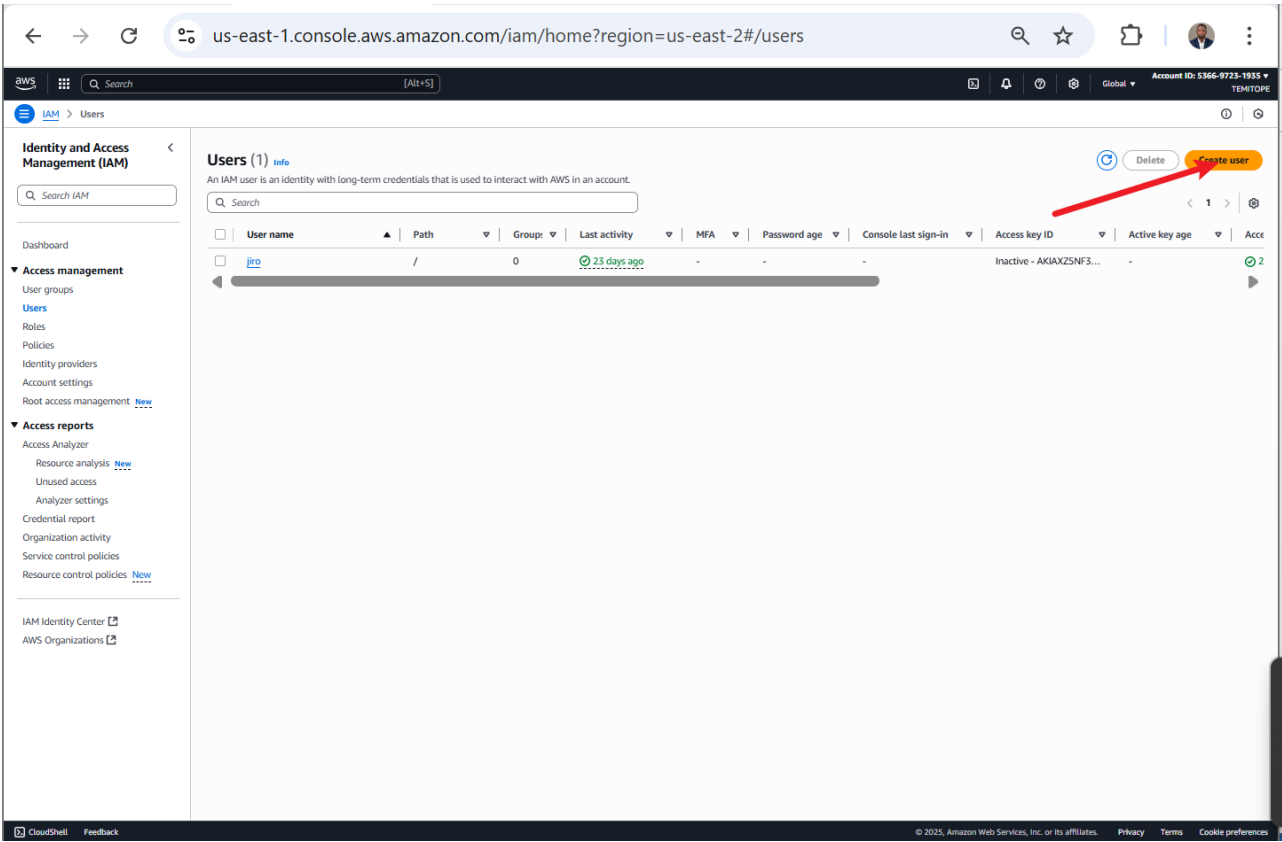
10. Policy Successfully Created



11. Click on Users, Proceed to Create User



12. On Users Page, Click Create User



13. Enter User Name, Give User Access to the Console, and Create User as an IAM User

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/users/create

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Step 4
Retrieve password

Specify user details

User details

User name

seun

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☒ Provide user access to the AWS Management Console - optional
If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Are you providing console access to a person?

User type

☐ Specify a user in Identity Center - Recommended
We recommend that you use Identity Center to provide console access to a person. With Identity Center, you can centrally manage user access to their AWS accounts and cloud applications.

☒ I want to create an IAM user
We recommend that you create IAM users only if you need to enable programmatic access through access keys, service-specific credentials for AWS CodeCommit or Amazon Keyspaces, or a backup credential for emergency account access.

14. Set Custom Password and Force User to Create New Password at Next Login and Click Next

If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Are you providing console access to a person?

User type

☐ Specify a user in Identity Center - Recommended
We recommend that you use Identity Center to provide console access to a person. With Identity Center, you can centrally manage user access to their AWS accounts and cloud applications.

☒ I want to create an IAM user
We recommend that you create IAM users only if you need to enable programmatic access through access keys, service-specific credentials for AWS CodeCommit or Amazon Keyspaces, or a backup credential for emergency account access.

Console password

☐ Autogenerated password
You can view the password after you create the user.

☒ Custom password
Enter a custom password for the user.

Seun@1234

- Must be at least 8 characters long
- Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } | ' "

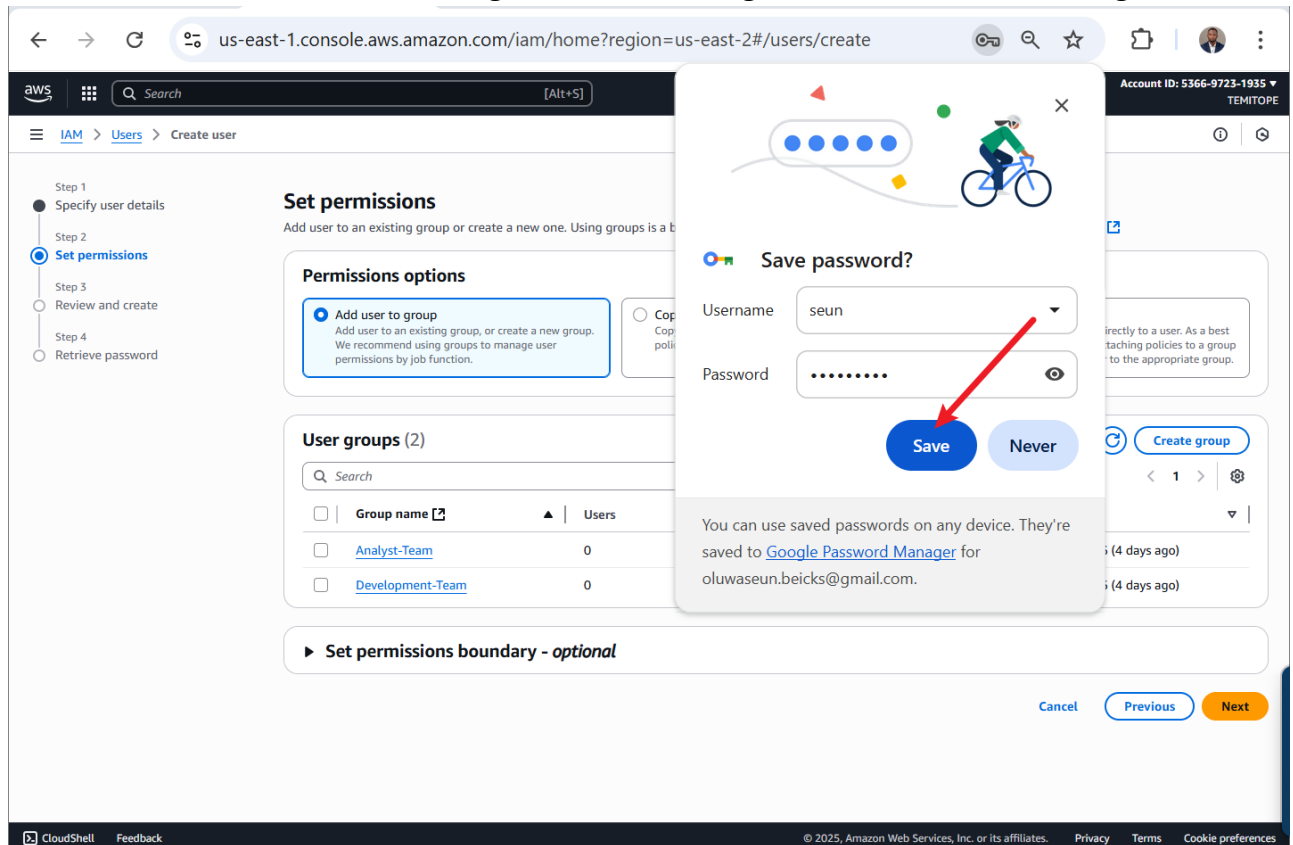
☒ Show password

☒ Users must create a new password at next sign-in - Recommended
Users automatically get the [IAMUserChangePassword](#) policy to allow them to change their own password.

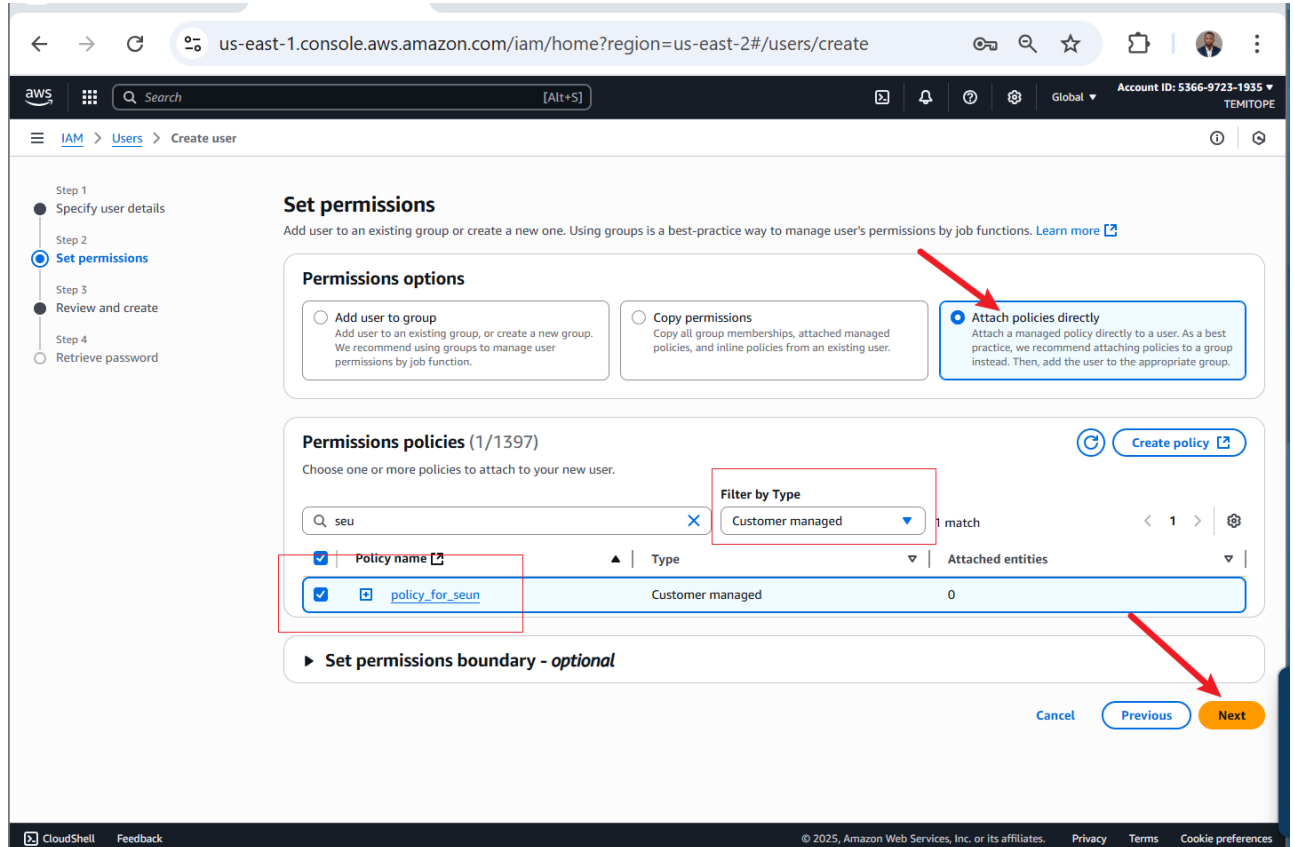
If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel Next

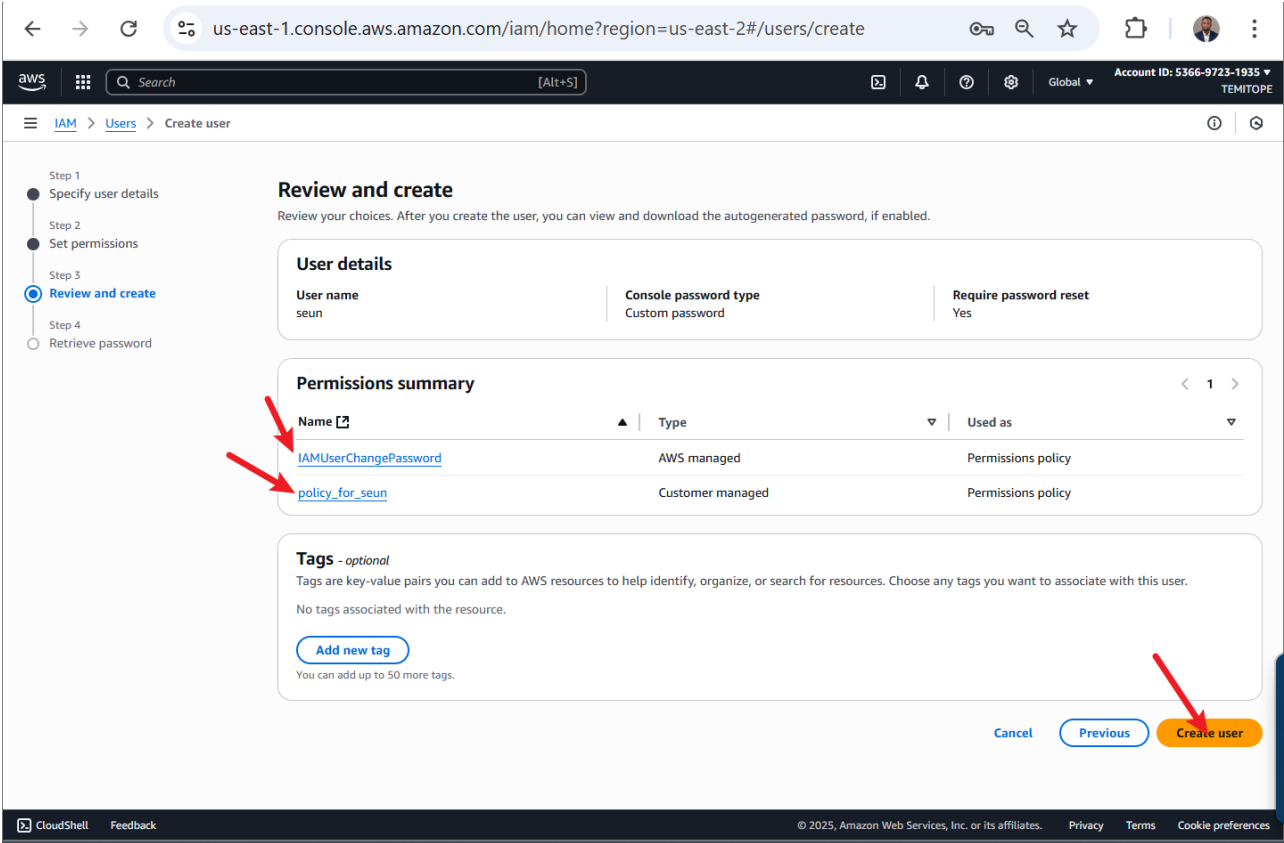
15. You Can Click Save Password on Google Password Manager If You Prefer for Next Log In



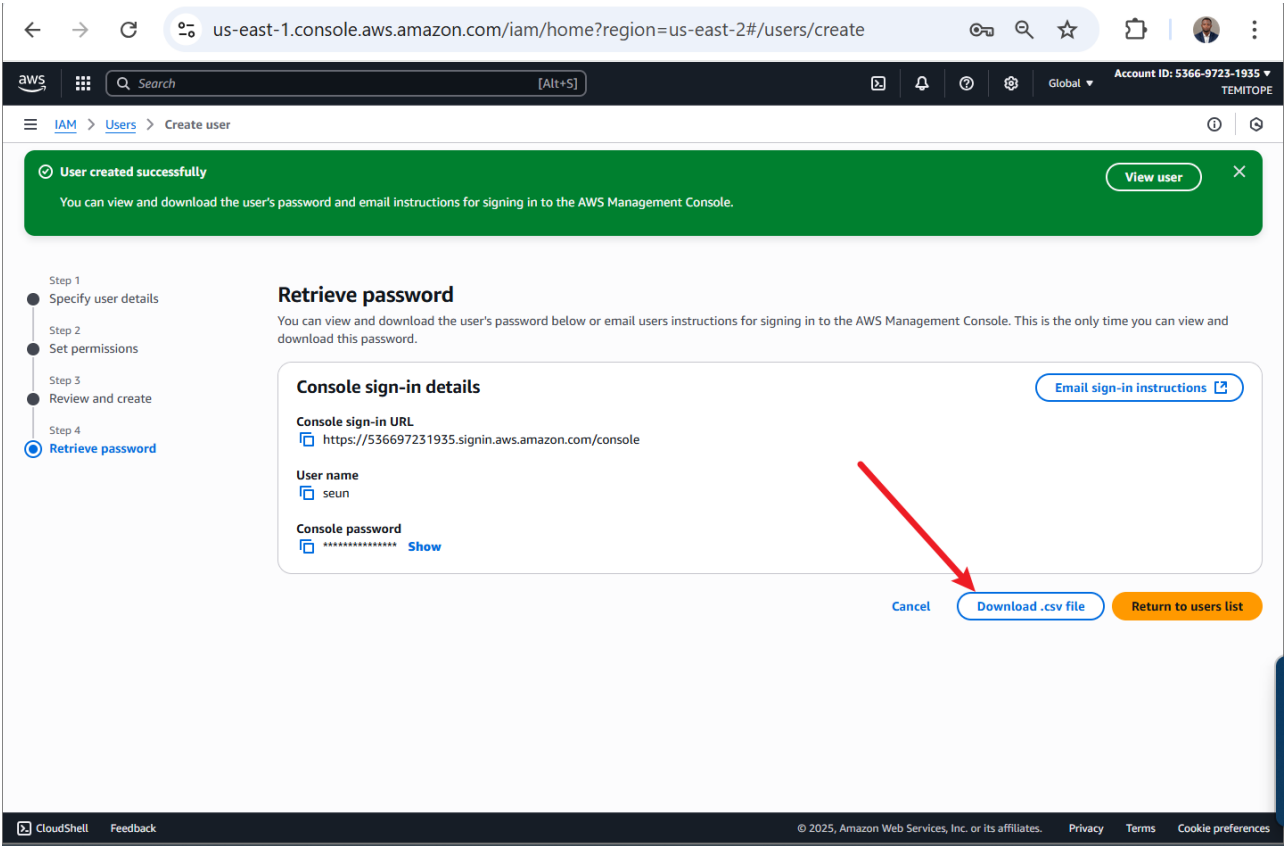
16. On Permission Page, Click Attach Policy Directly, Filter Policy by Customer Managed Policy, Select Created User Policy, and Click Next



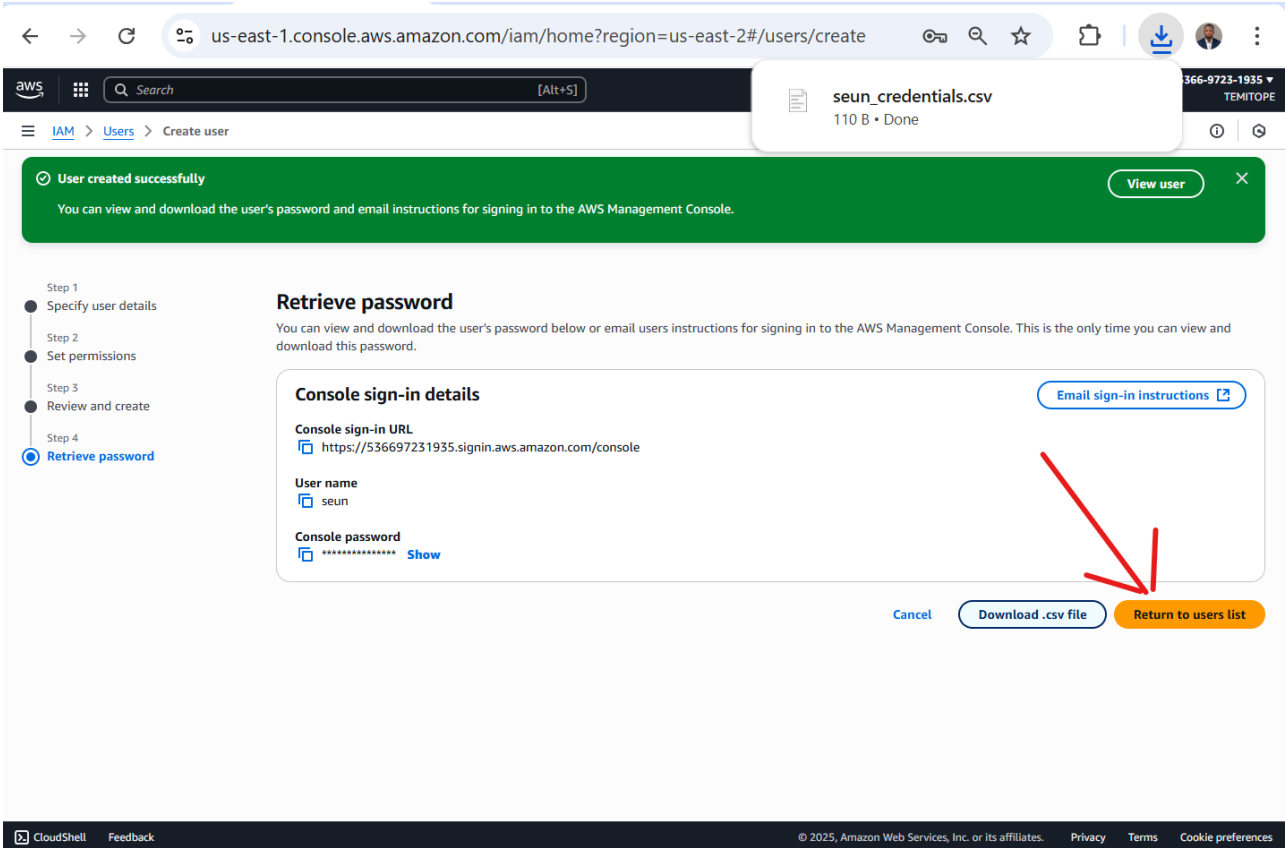
17. Review and Create



18. User Seun Successfully Created, Now Download CSV File to Retrieve Chosen Password

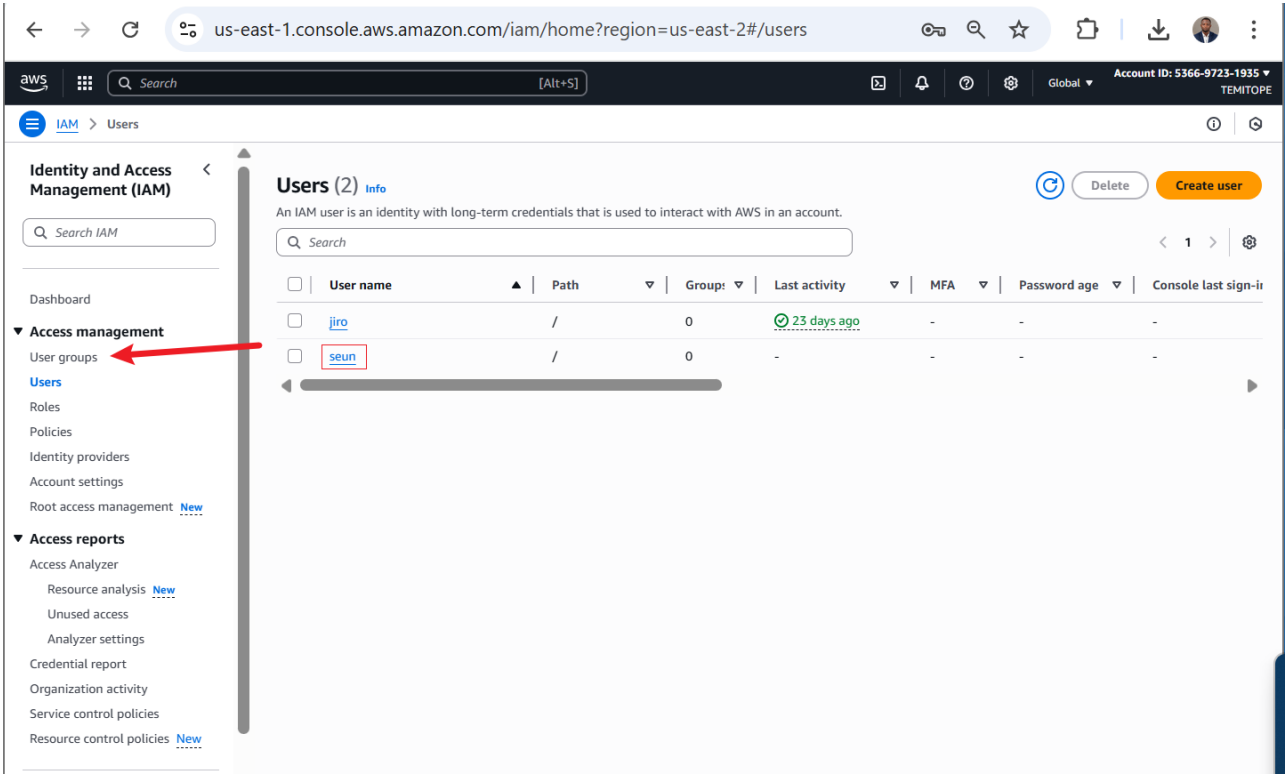


19. Credential Successfully Downloaded, Proceed to User List Page

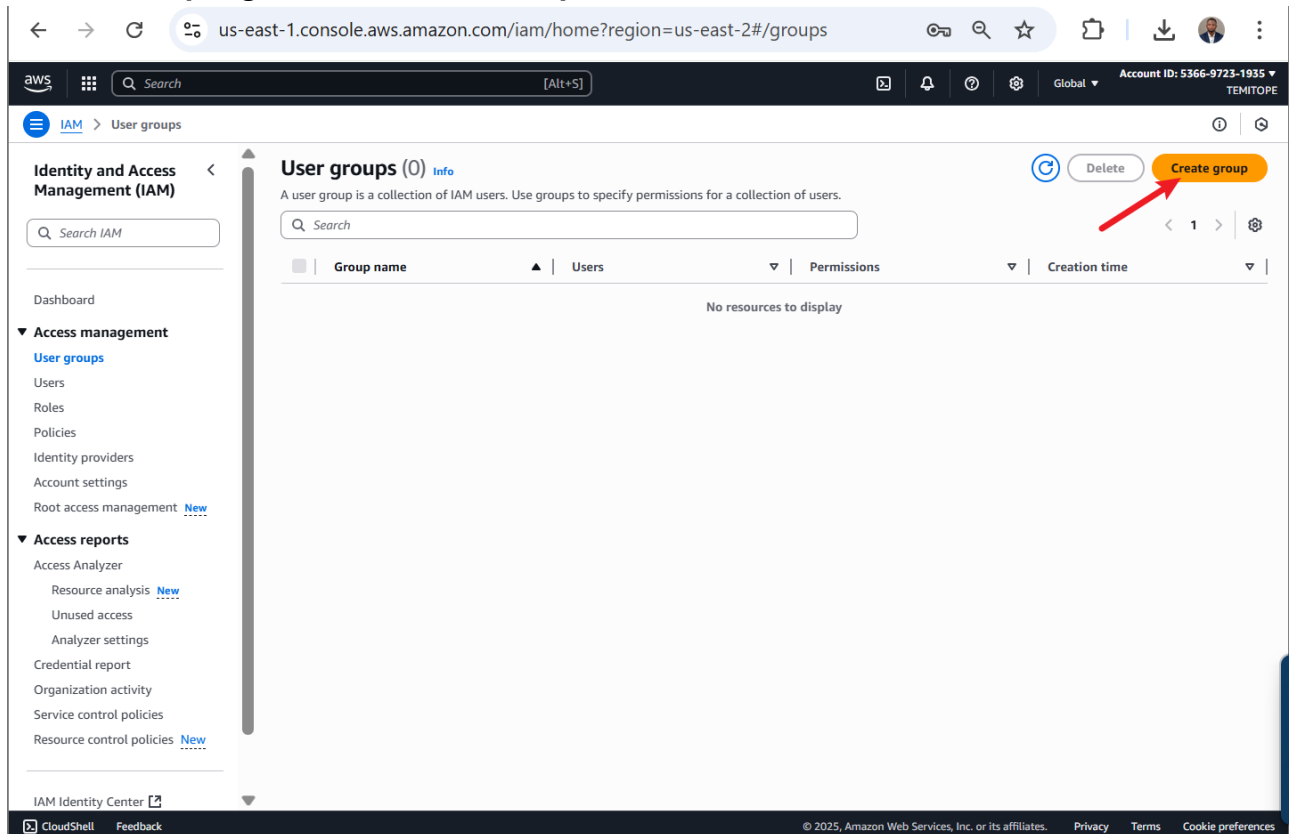


Part 2: Create Group, Add Users (Jack and Ade), Create EC2/S3 Policy, and Attach to Group

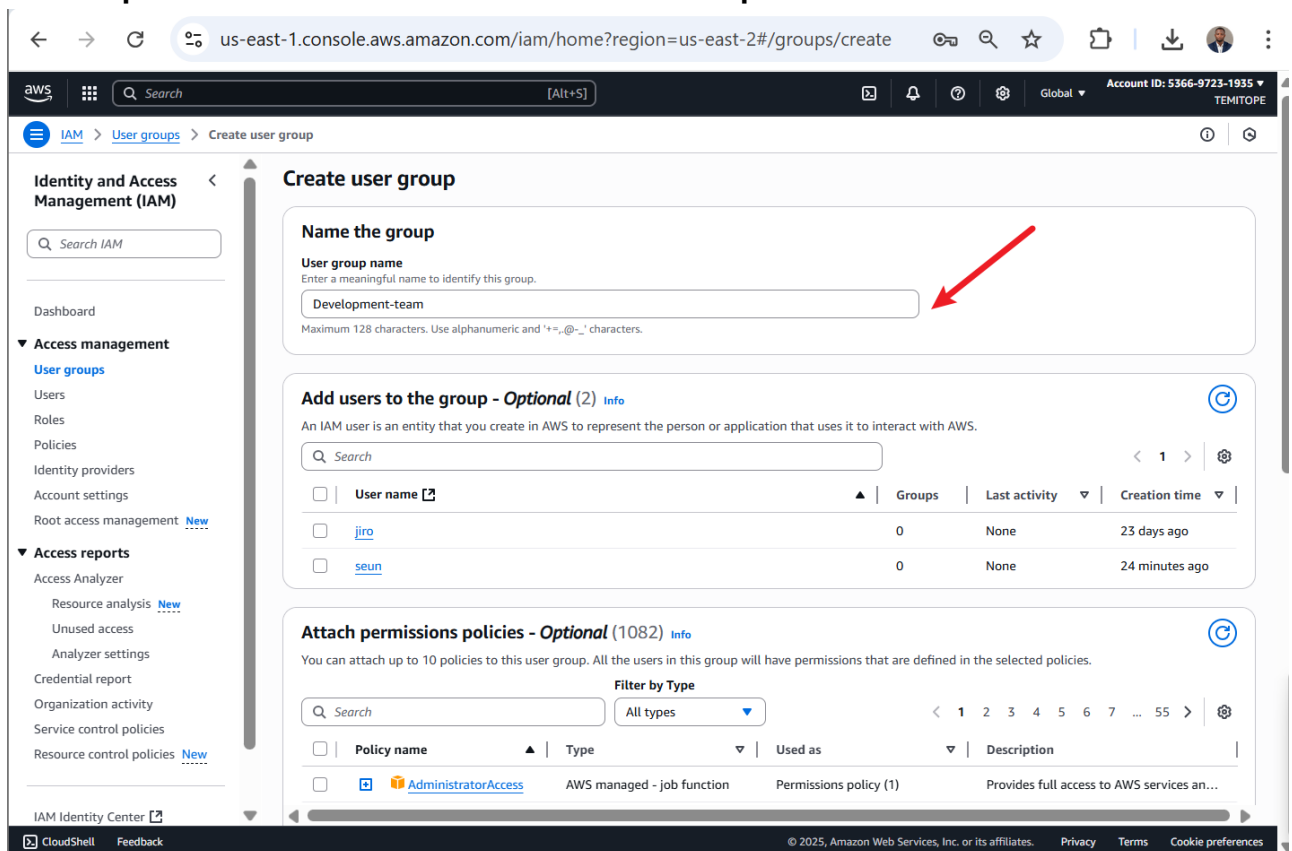
20. Created User Listed, Now Click on User Group



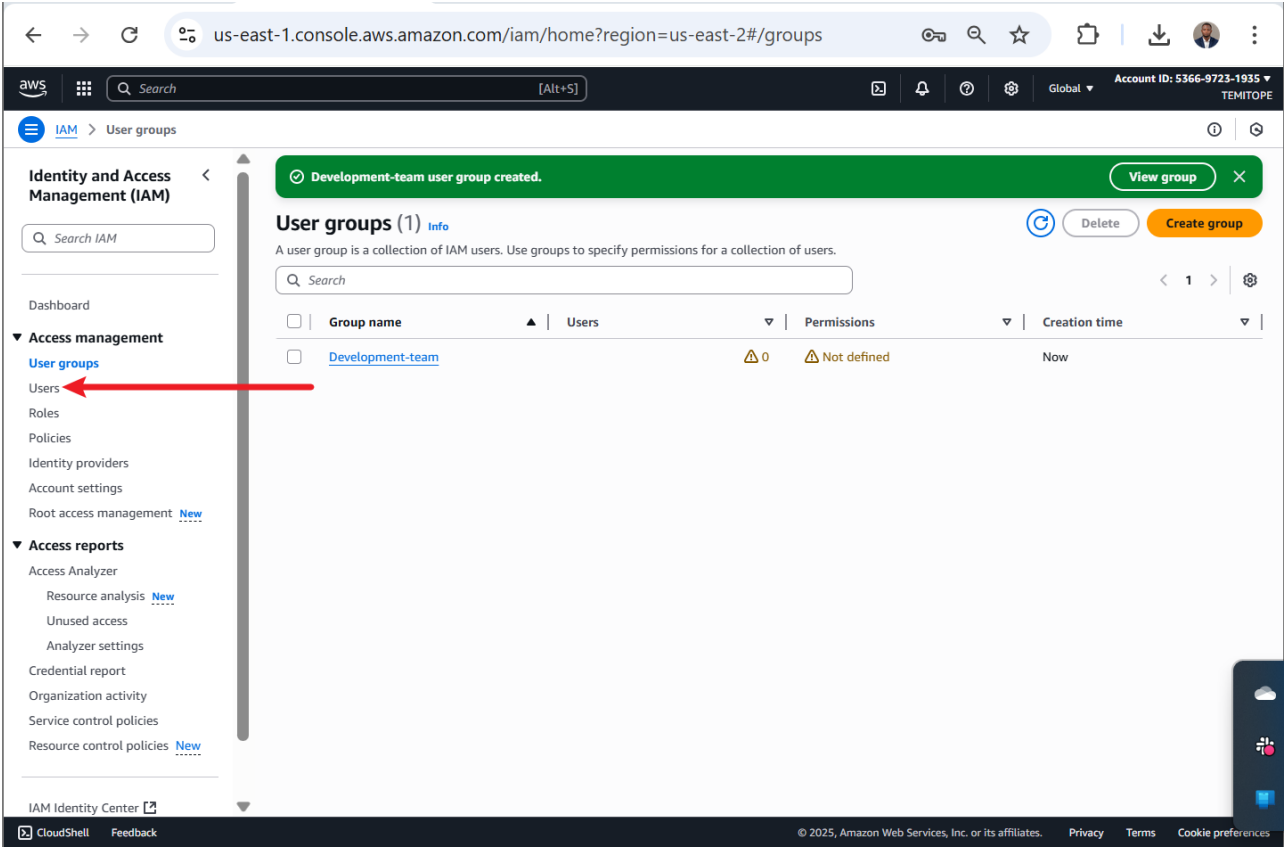
21. On User Group Page, Click on Create Group



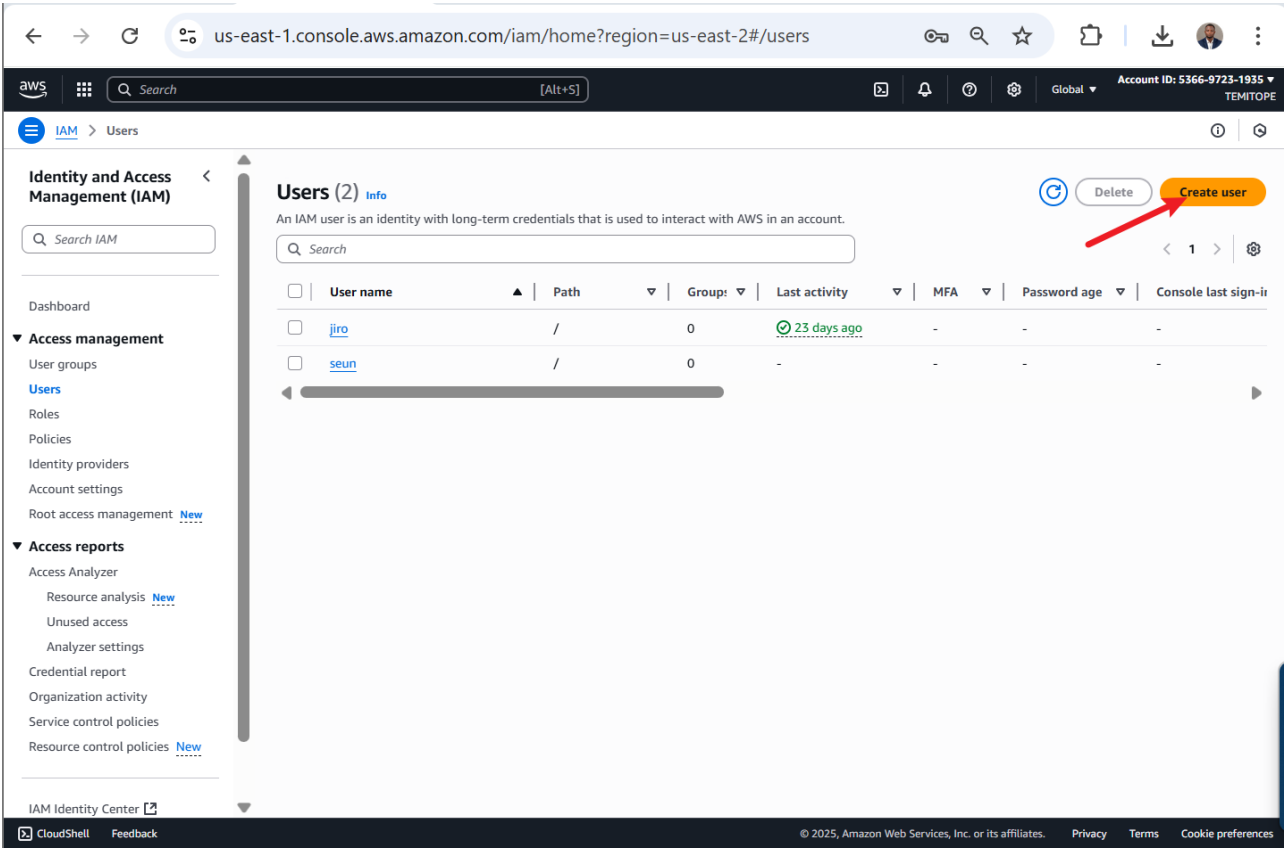
22. Set Group Name and Scroll Down and Click Create Group



23. Group Successfully Created, Click Users to Create New User



24. User List Page Appears, Click Create User



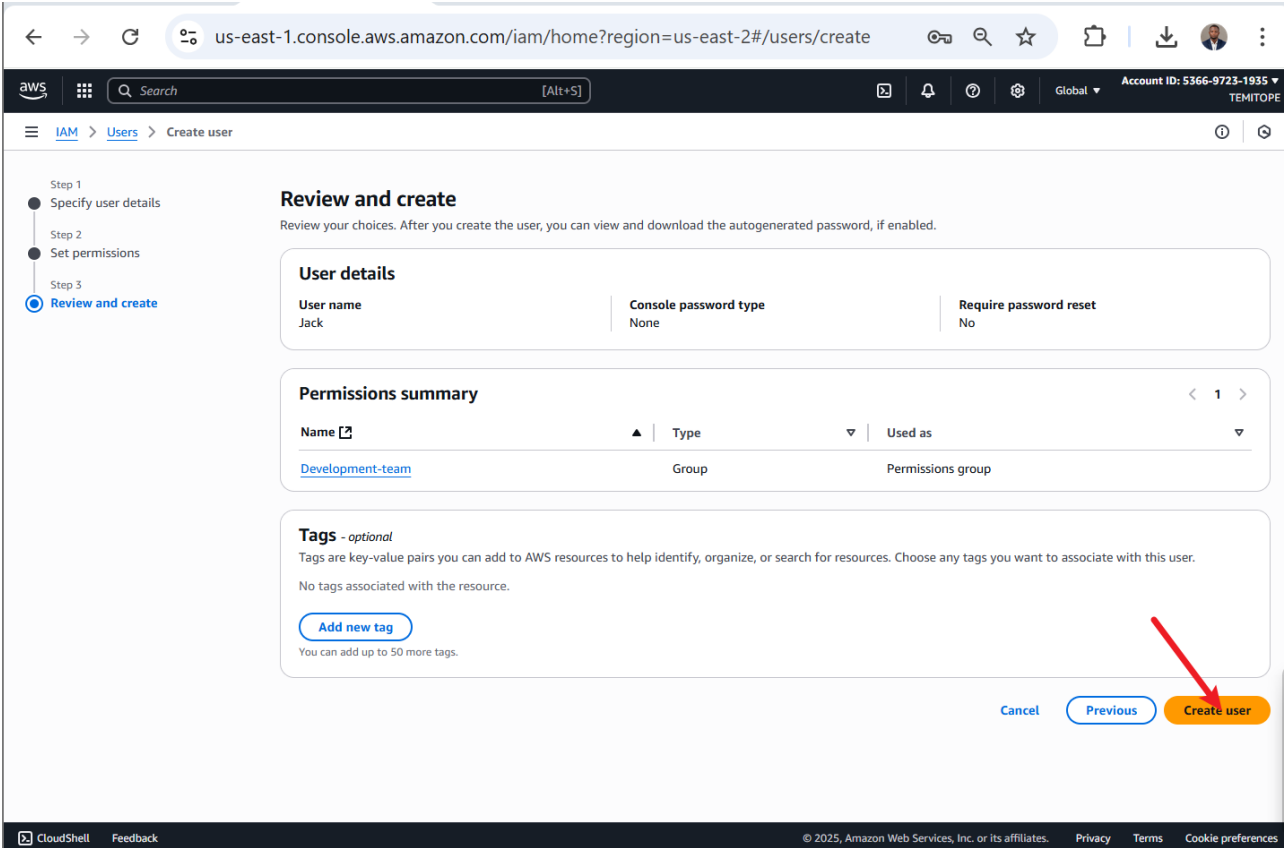
25. Choose User Name and Click Next

The screenshot shows the 'Specify user details' step in the AWS IAM console. The 'User name' field is populated with 'Jack'. A red arrow points to this field. Below the field, there is a checkbox for 'Provide user access to the AWS Management Console - optional'. A blue information box at the bottom states: 'If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)'. At the bottom right, there are 'Cancel' and 'Next' buttons. A red arrow points to the 'Next' button. The breadcrumb trail at the top reads 'IAM > Users > Create user'.

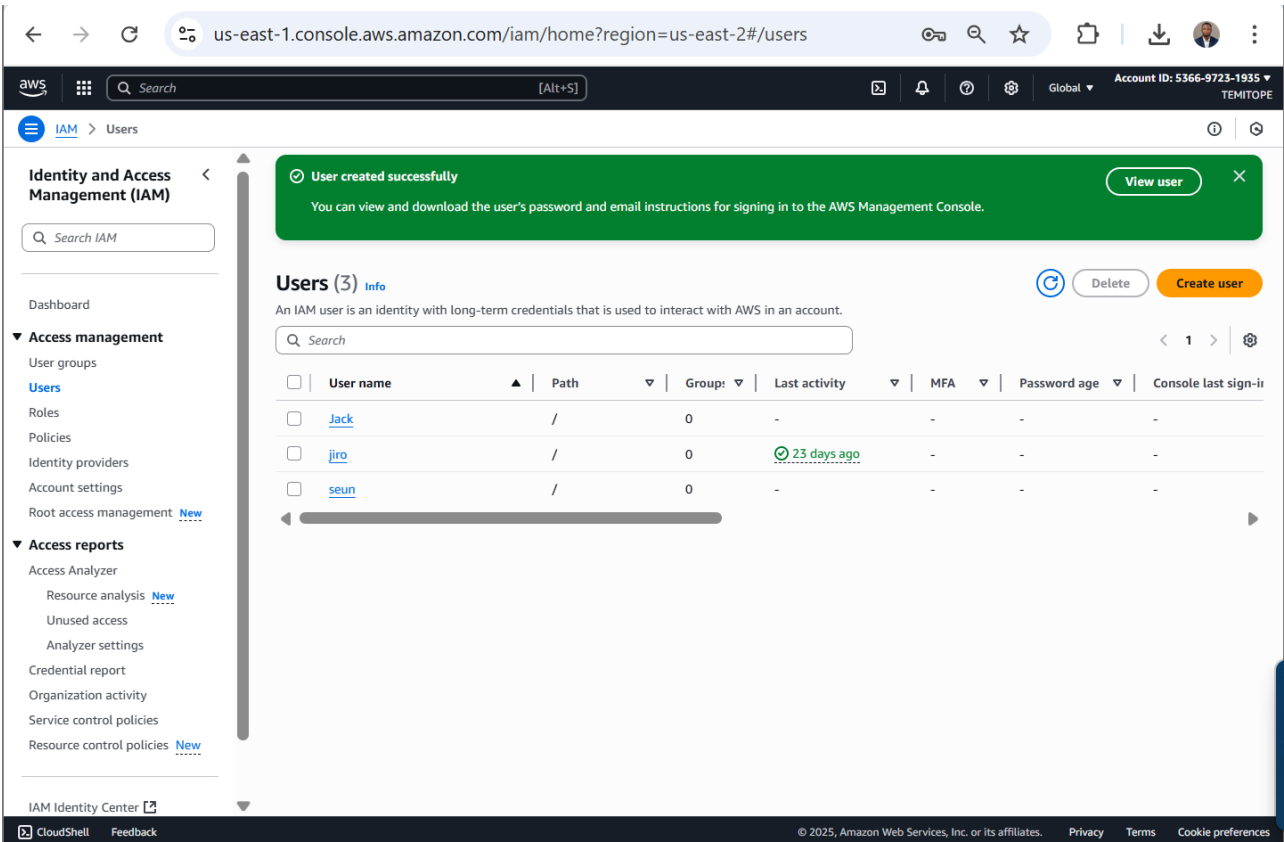
26. In the Permission Option, Select Add User to Group and Select Development-Team and Click Next

The screenshot shows the 'Set permissions' step in the AWS IAM console. The 'Permissions options' section has three radio buttons: 'Add user to group' (selected), 'Copy permissions', and 'Attach policies directly'. A red arrow points to the 'Add user to group' option. Below this, the 'User groups (1/1)' section shows a table with one group, 'Development-team', which is selected with a checkbox. A red arrow points to this checkbox. The table has columns for 'Group name', 'Users', 'Attached policies', and 'Created'. At the bottom right, there are 'Cancel', 'Previous', and 'Next' buttons. A red arrow points to the 'Next' button. The breadcrumb trail at the top reads 'IAM > Users > Create user'.

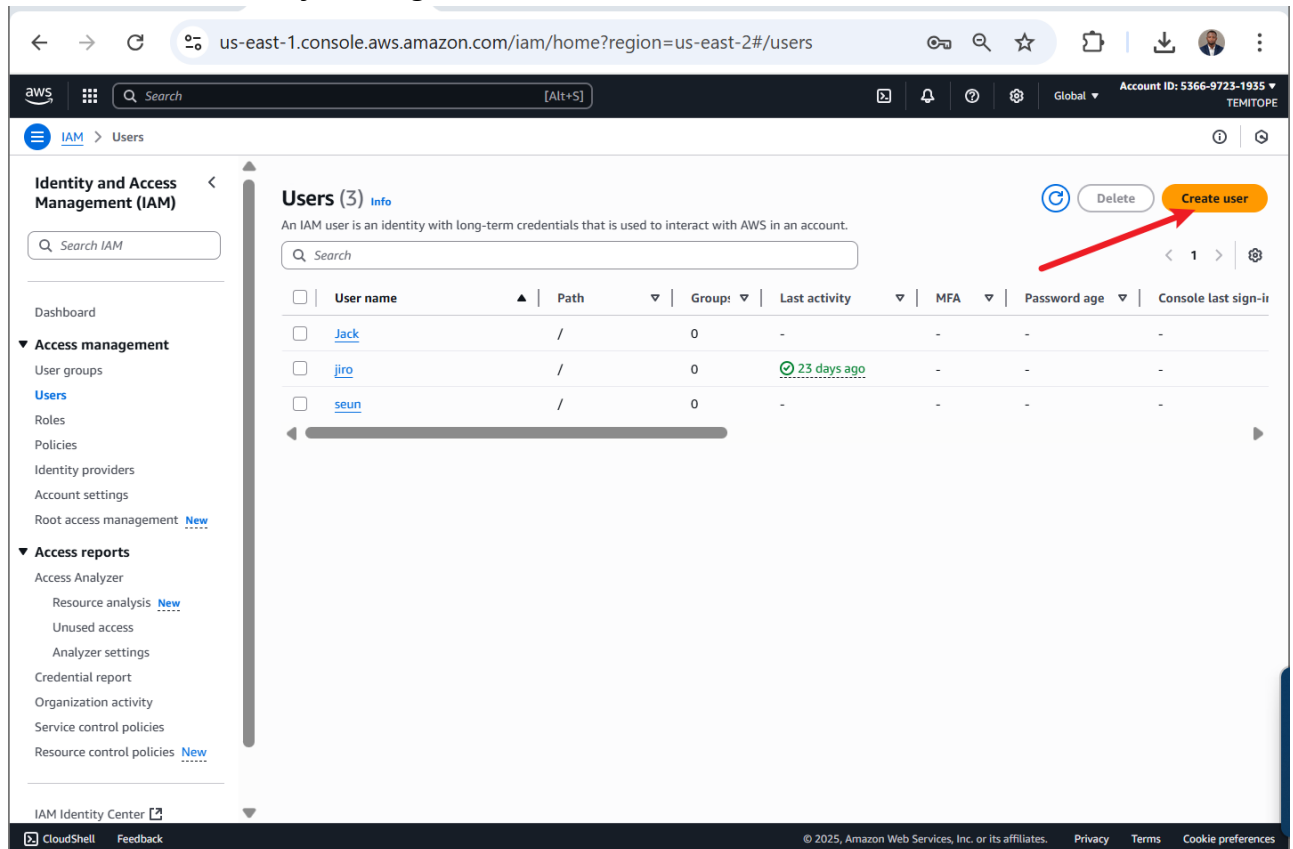
27. Review and Click Create User



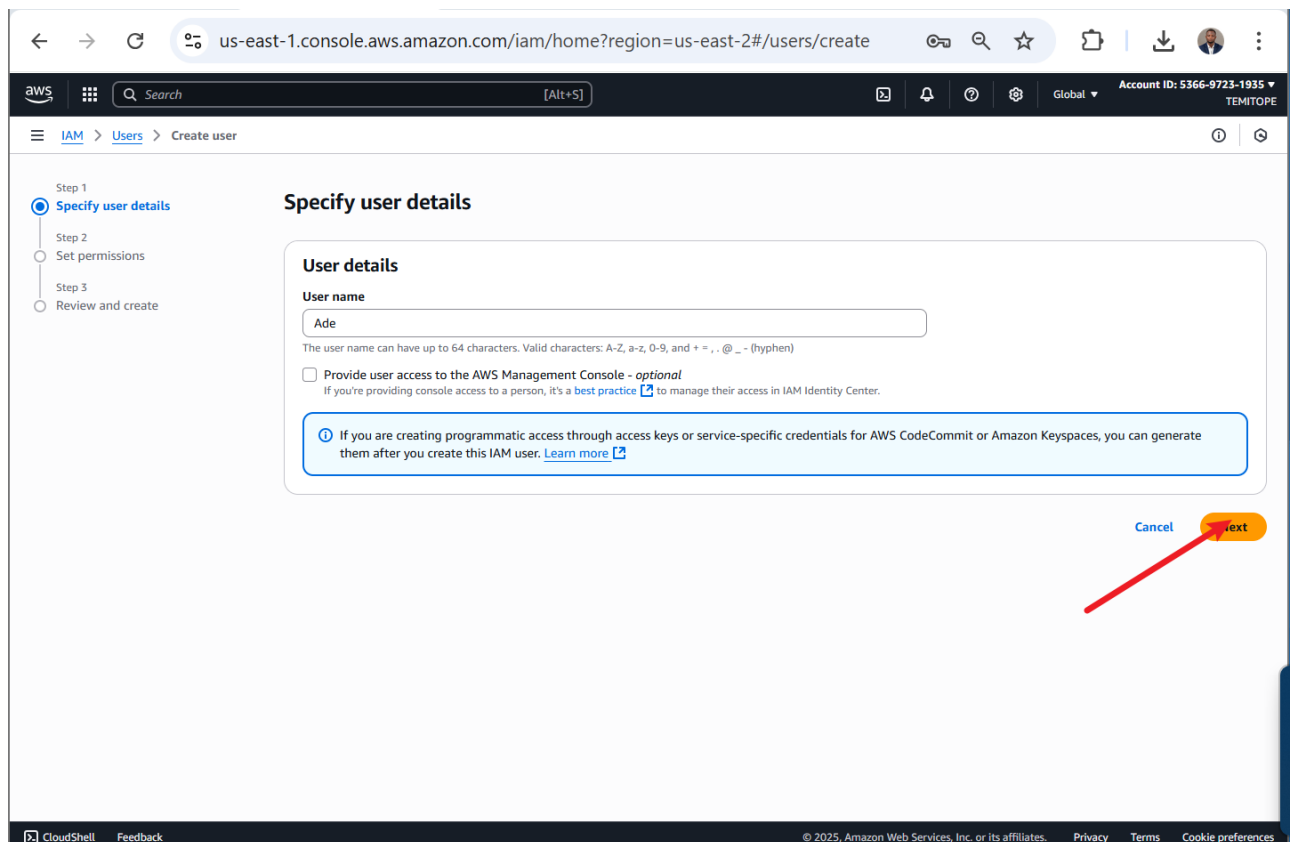
28. User Jack Successfully Created



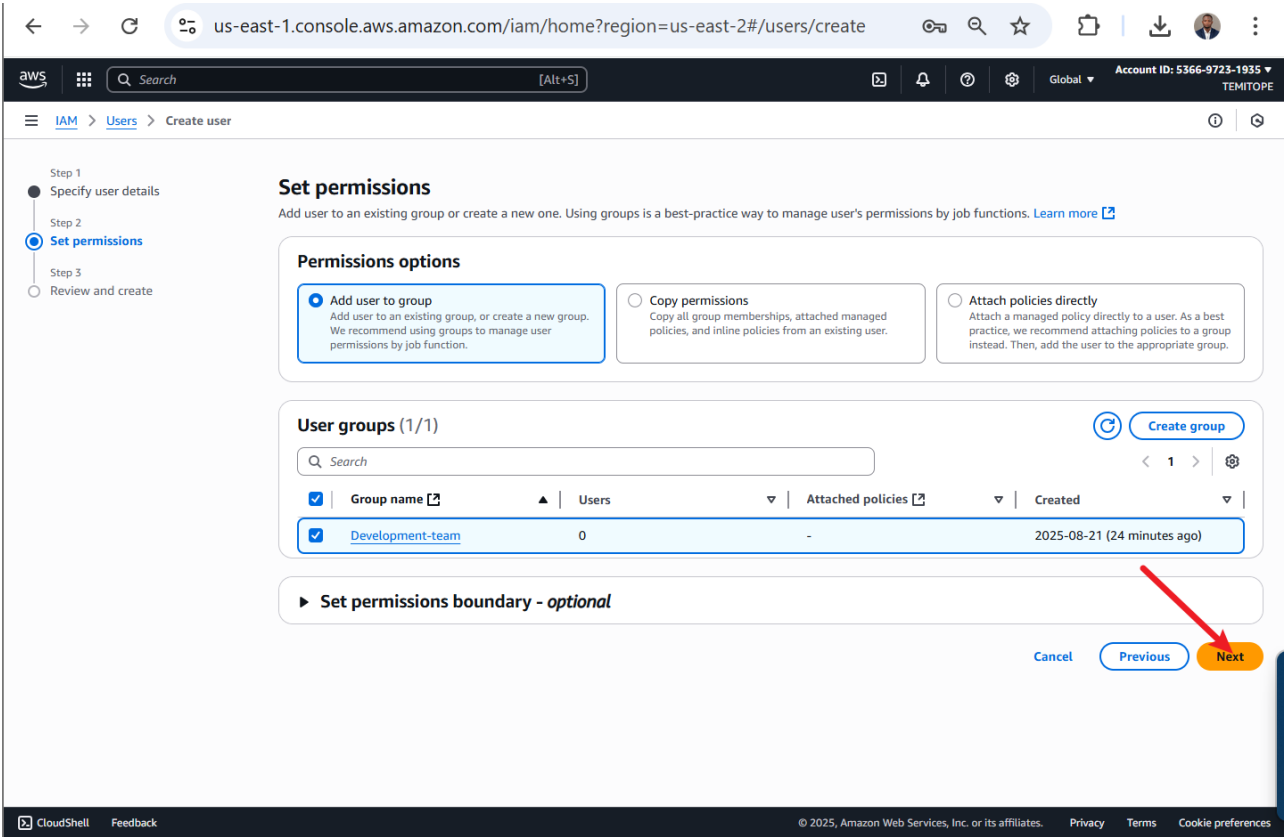
29. Create Another User by Clicking Create User



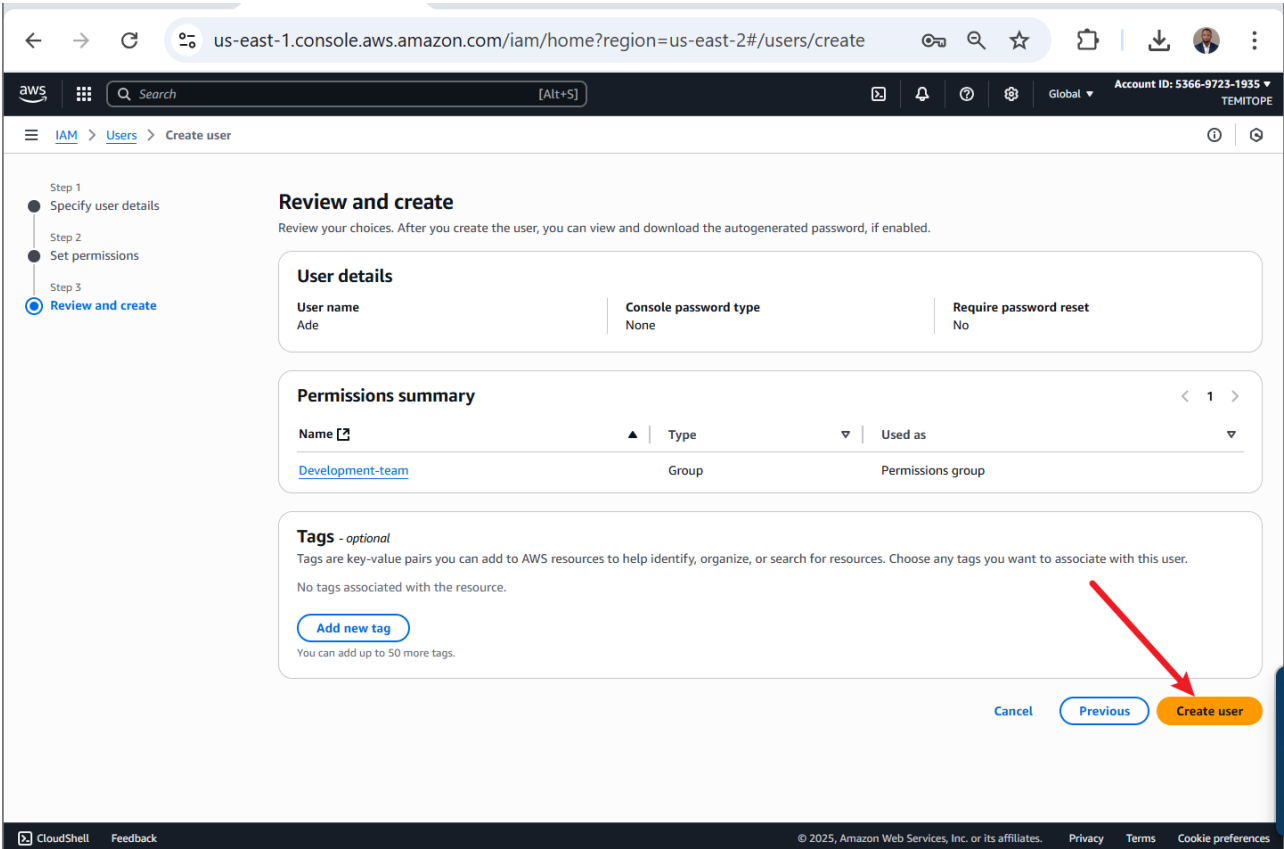
30. Choose User Name and Click Next



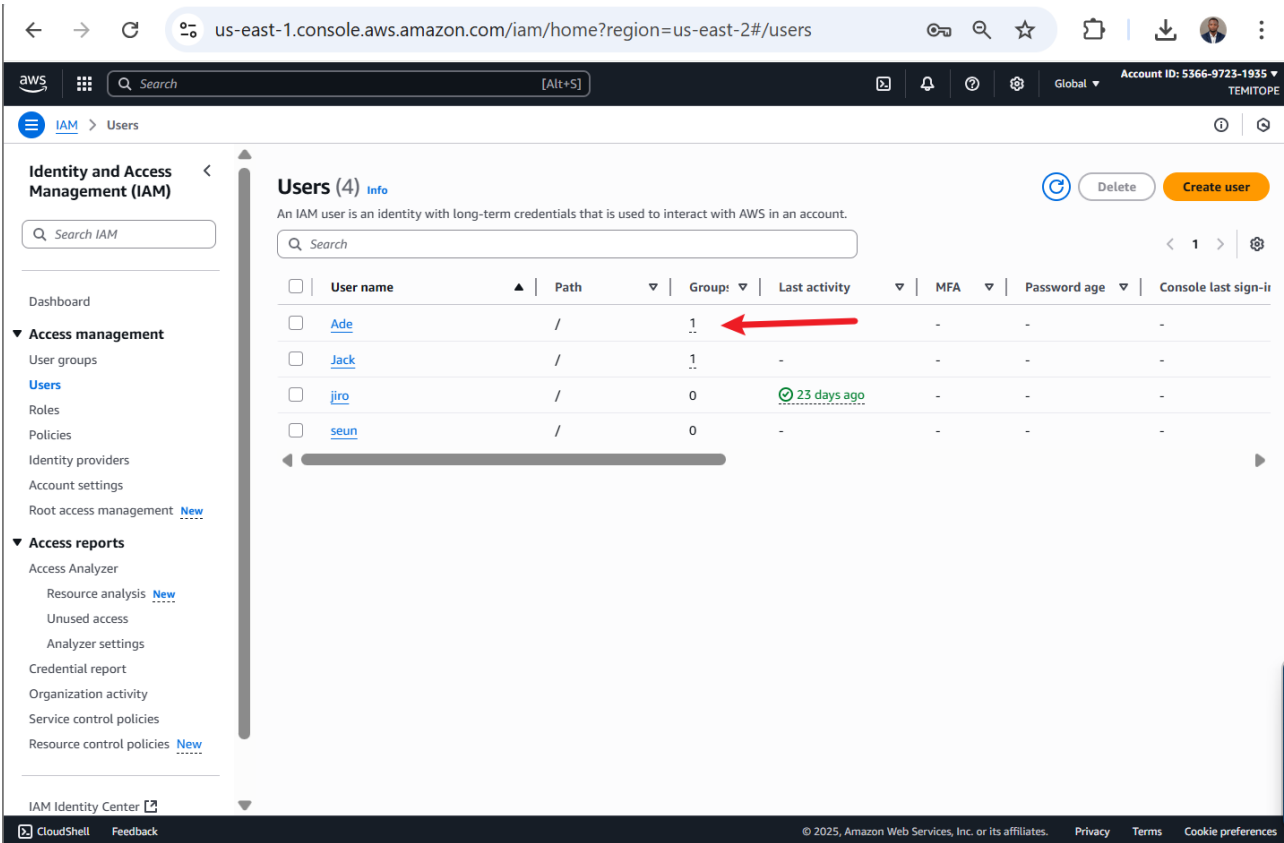
31. In the Permission Option, Select Add User to Group and Select Development-Team and Click Next



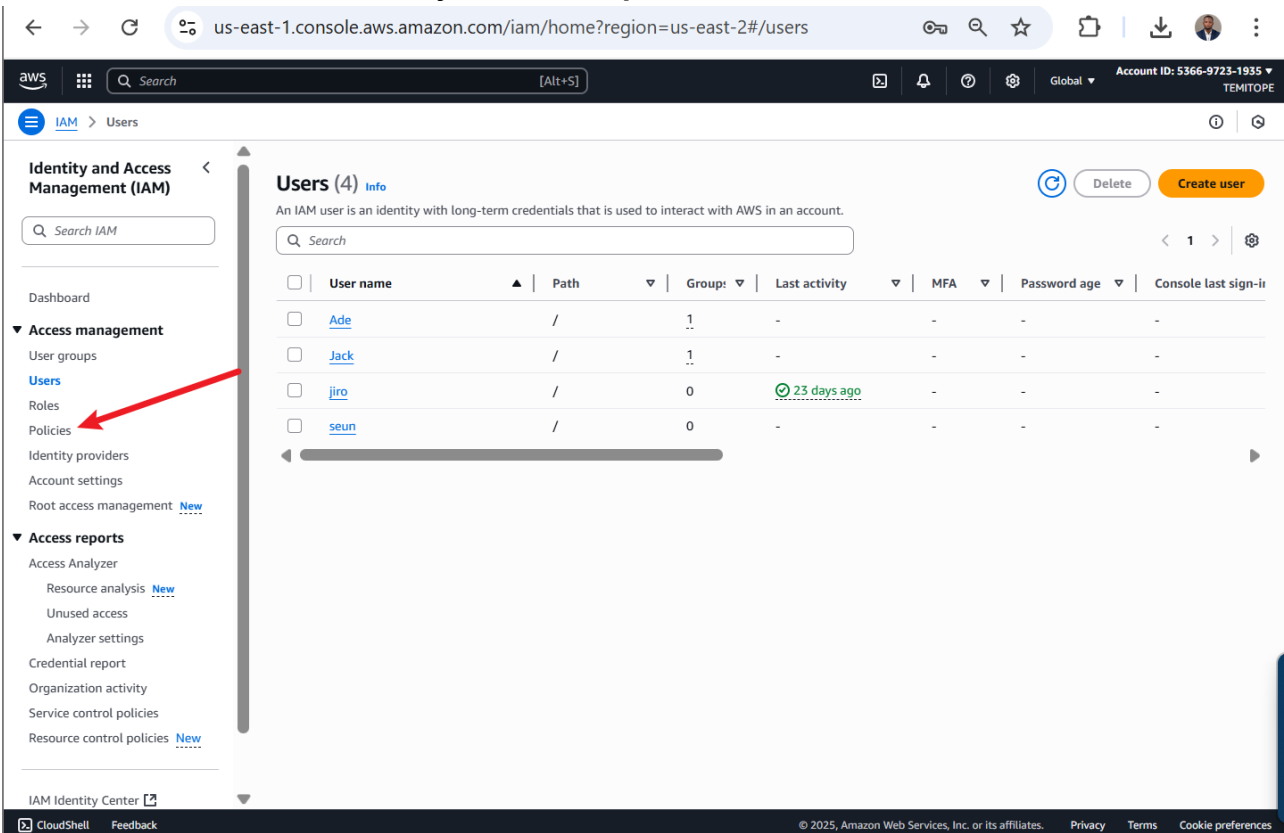
32. Review and Click Create User



33. User Ade Successfully Created



34. Click Policies to Create New Policy for the Group



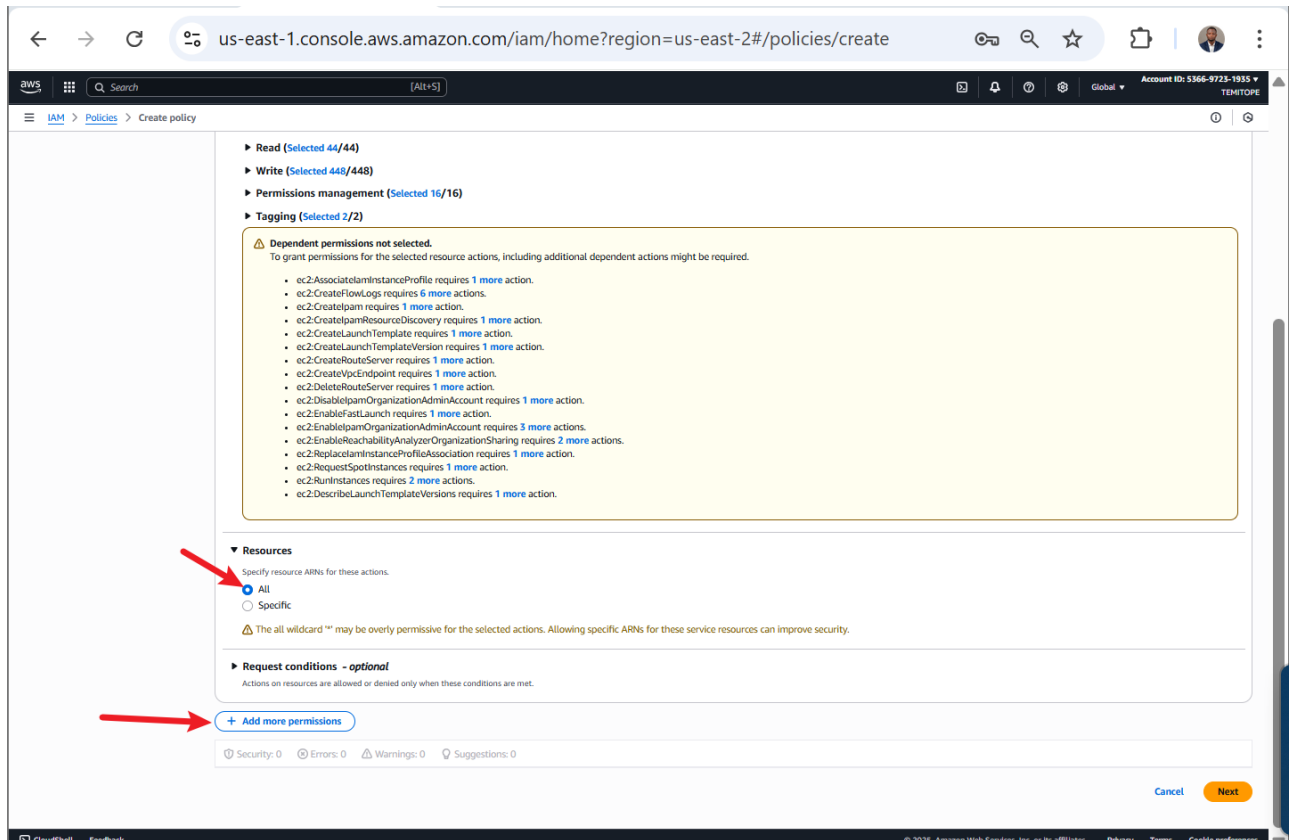
35. Click Create Policy

The screenshot shows the AWS IAM console 'Policies' page. The breadcrumb navigation is 'IAM > Policies'. The page title is 'Policies (1397) Info'. A description states: 'A policy is an object in AWS that defines permissions.' There are buttons for 'Actions', 'Delete', and 'Create policy'. The 'Create policy' button is highlighted with a red arrow. Below the buttons is a table of existing policies. The table has columns: Policy name, Type, Used as, and Description. The table lists various AWS managed policies, including 'AccessAnalyzerService...', 'AdministratorAccess', 'AIOPSAssistantPolicy', 'AIOPSConsoleAdmin...', 'AIOPSOperatorAccess', 'AIOPSReadOnlyAccess', 'AlexaForBusinessDev...', 'AlexaForBusinessFull...', 'AlexaForBusinessGat...', 'AlexaForBusinessLife...', 'AlexaForBusinessNet...', 'AlexaForBusinessPol...', and 'AlexaForBusinessRes...'. The 'Create policy' button is located at the top right of the table.

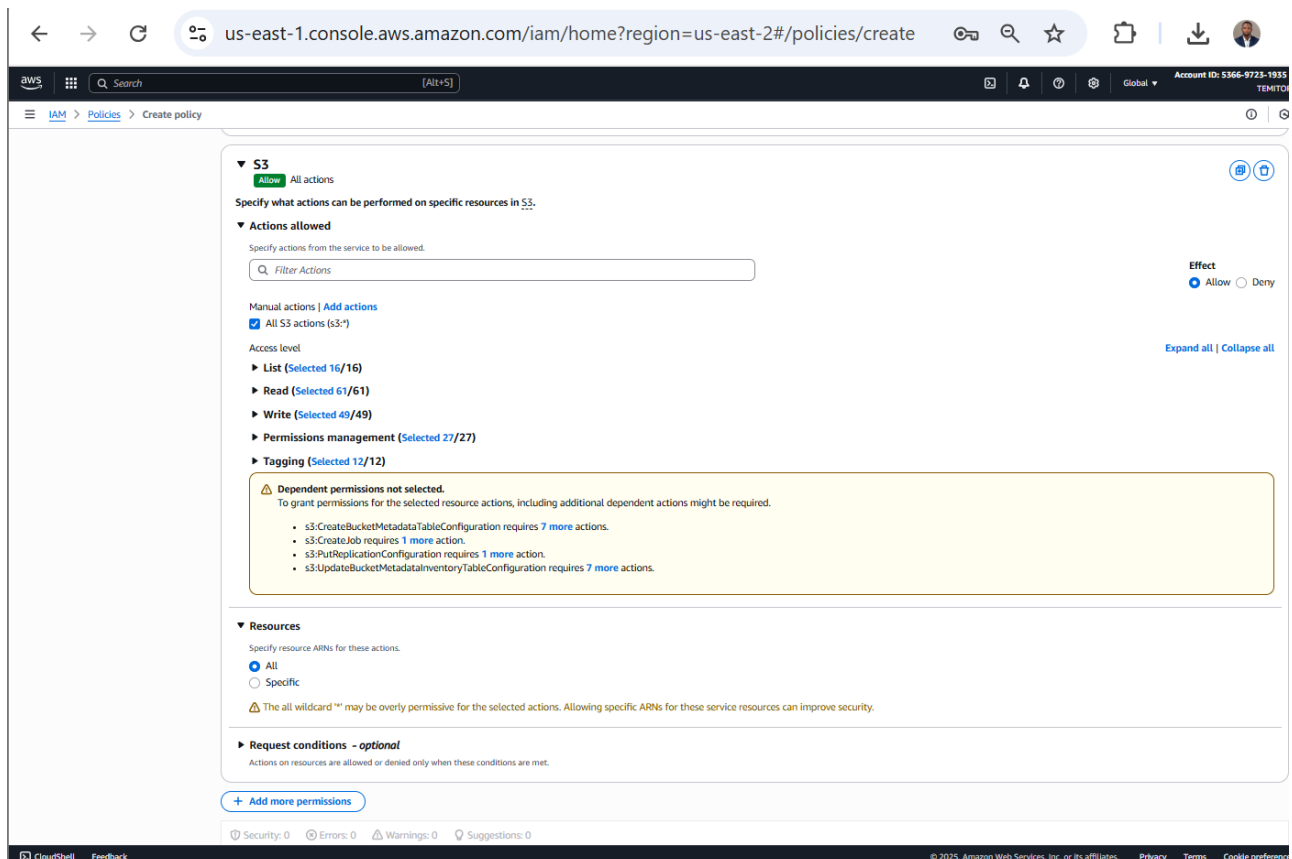
36. Choose EC2 and Select All Actions and Resources

The screenshot shows the AWS IAM console 'Create policy' page. The breadcrumb navigation is 'IAM > Policies > Create policy'. The page title is 'Specify permissions Info'. A description states: 'Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.' There are buttons for 'Visual', 'JSON', 'Actions', and 'Add'. The 'EC2' service is selected. Under 'Actions allowed', 'All EC2 actions (ec2*)' is selected. A red arrow points to the 'All EC2 actions (ec2*)' checkbox. Below the 'Actions allowed' section, there is a list of 'Access level' actions: 'List (Selected 194/194)', 'Read (Selected 44/44)', 'Write (Selected 448/448)', 'Permissions management (Selected 16/16)', and 'Tagging (Selected 2/2)'. A warning box at the bottom states: 'Dependent permissions not selected. To grant permissions for the selected resource actions, including additional dependent actions might be required.' The warning box lists several dependent actions, including 'ec2:AssociateInstanceProfile', 'ec2:CreateFlowLogs', 'ec2:CreateIam', 'ec2:CreateIamResourceDiscovery', 'ec2:CreateLaunchTemplate', 'ec2:CreateLaunchTemplateVersion', 'ec2:CreateRouteServer', 'ec2:CreateVpcEndpoint', 'ec2:DeleteRouteServer', 'ec2:DisableIamOrganizationAdminAccount', 'ec2:EnableFastLaunch', 'ec2:EnableIamOrganizationAdminAccount', 'ec2:EnableReachabilityAnalyzerOrganizationSharing', 'ec2:ReplaceInstanceProfileAssociation', 'ec2:RequestSpotInstances', and 'ec2:RunInstances'.

37. Select Add More Permission and Select S3 Service



38. Choose S3 and Select All Actions and All Resources and Click Next



39. Name Policy and Click Create Policy

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/policies/create

Step 1: Specify permissions
Step 2: Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name
Enter a meaningful name to identify this policy.
development-team-policy
Maximum 128 characters. Use alphanumeric and "+, -, @, _" characters.

Description - optional
Add a short explanation for this policy.
Maximum 1,000 characters. Use alphanumeric and "+, -, @, _" characters.

Permissions defined in this policy
Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it.

Search

Allow (2 of 450 services) Show remaining 448 services

Service	Access level	Resource	Request condition
EC2	Full access	All resources	None
S3	Full access	All resources	None

Add tags - optional
Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.
No tags associated with the resource.

Add new tag
You can add up to 50 more tags.

Cancel Previous **Create policy**

40. Development-Team-Policy Successfully Created, Navigate to User Group Page

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/policies

Policy development-team-policy created. View policy

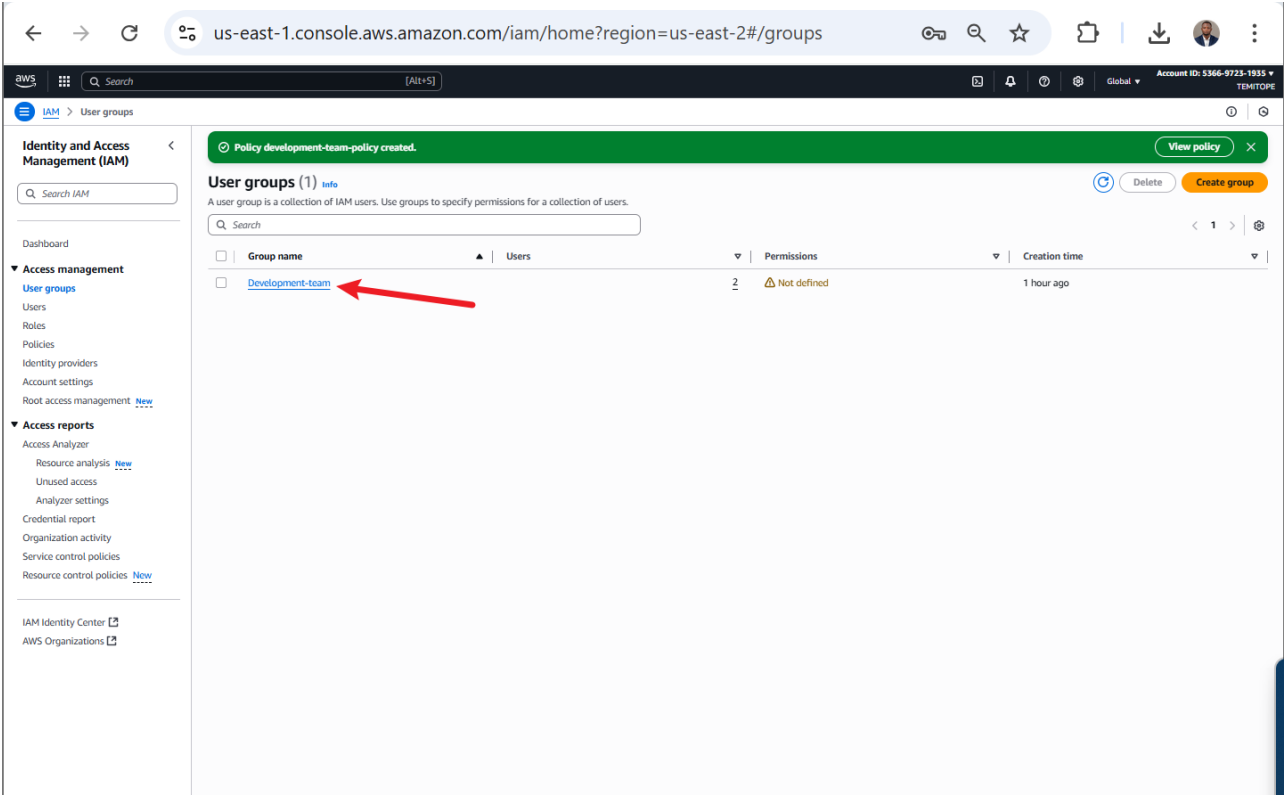
Policies (1398)

A policy is an object in AWS that defines permissions.

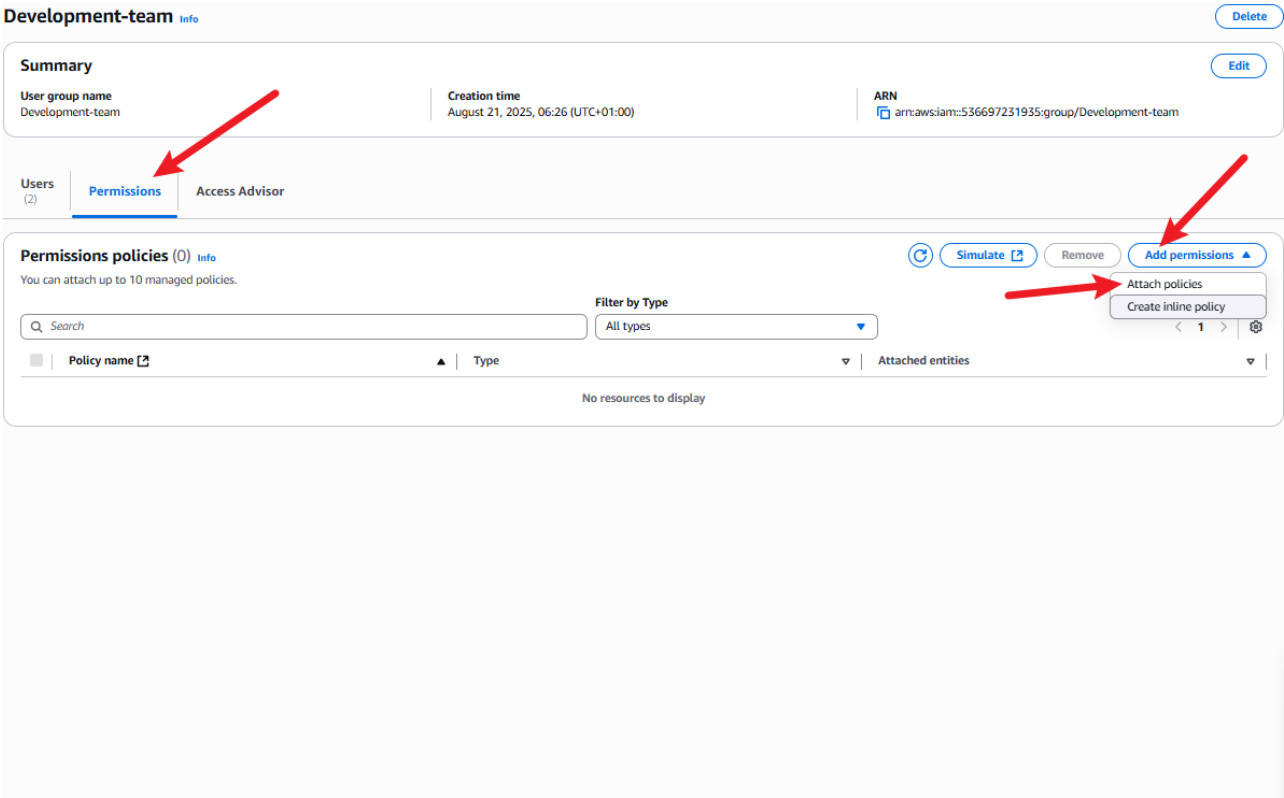
Search Filter by Type All types

Policy name	Type	Used as	Description
AccessAnalyzerServiceRolePolicy	AWS managed	None	Allow Access Analyzer to analyze resou.
AdministratorAccess	AWS managed - job function	Permissions policy (1)	Provides full access to AWS services an.
AdministratorAccess-Amplify	AWS managed	None	Grants account administrative permisi.
AdministratorAccess-AWSElasticBeanstalk	AWS managed	None	Grants account administrative permisi.
AIAssistantPolicy	AWS managed	None	Provides ReadOnly permissions require.
AIConsoleAdminPolicy	AWS managed	None	Grants full access to Amazon AI Opera..
AIOperatorAccess	AWS managed	None	Grants access to the Amazon AI Opera..
AIReadOnlyAccess	AWS managed	None	Grants ReadOnly permissions to the A..
AlexaForBusinessDeviceSetup	AWS managed	None	Provide device setup access to AlexaFo.
AlexaForBusinessFullAccess	AWS managed	None	Grants full access to AlexaForBusiness ..
AlexaForBusinessGatewayExecution	AWS managed	None	Provide gateway execution access to AL.
AlexaForBusinessLifesizeDelegatedAccessPolicy	AWS managed	None	Provide access to Lifesize AWS devices
AlexaForBusinessNetworkProfileServicePolicy	AWS managed	None	This policy enables Alexa for Business ..
AlexaForBusinessPolyDelegatedAccessPolicy	AWS managed	None	Provide access to Poly AWS devices
AlexaForBusinessReadOnlyAccess	AWS managed	None	Provide read only access to AlexaForBu.
AmazonAPIGatewayAdministrator	AWS managed	None	Provides full access to create/edit/dele.
AmazonAPIGatewayInvokeFullAccess	AWS managed	None	Provides full access to invoke APIs in A..
AmazonAPIGatewayPushToCloudWatchLogs	AWS managed	None	Allows API Gateway to push logs to us..
AmazonAppFlowFullAccess	AWS managed	None	Provides full access to Amazon AppFlo..
AmazonAppFlowReadOnlyAccess	AWS managed	None	Provides read-only access to Amazon A.

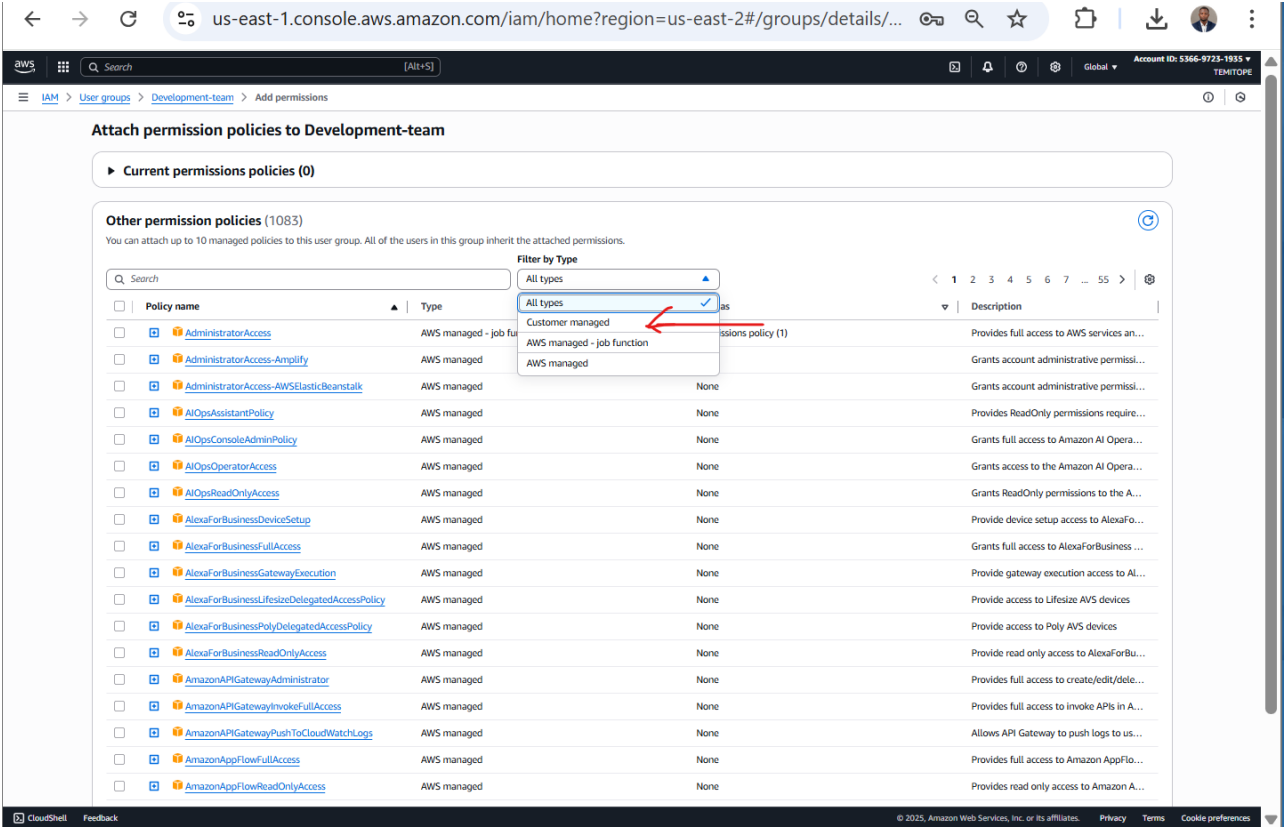
41. Click Development-Team



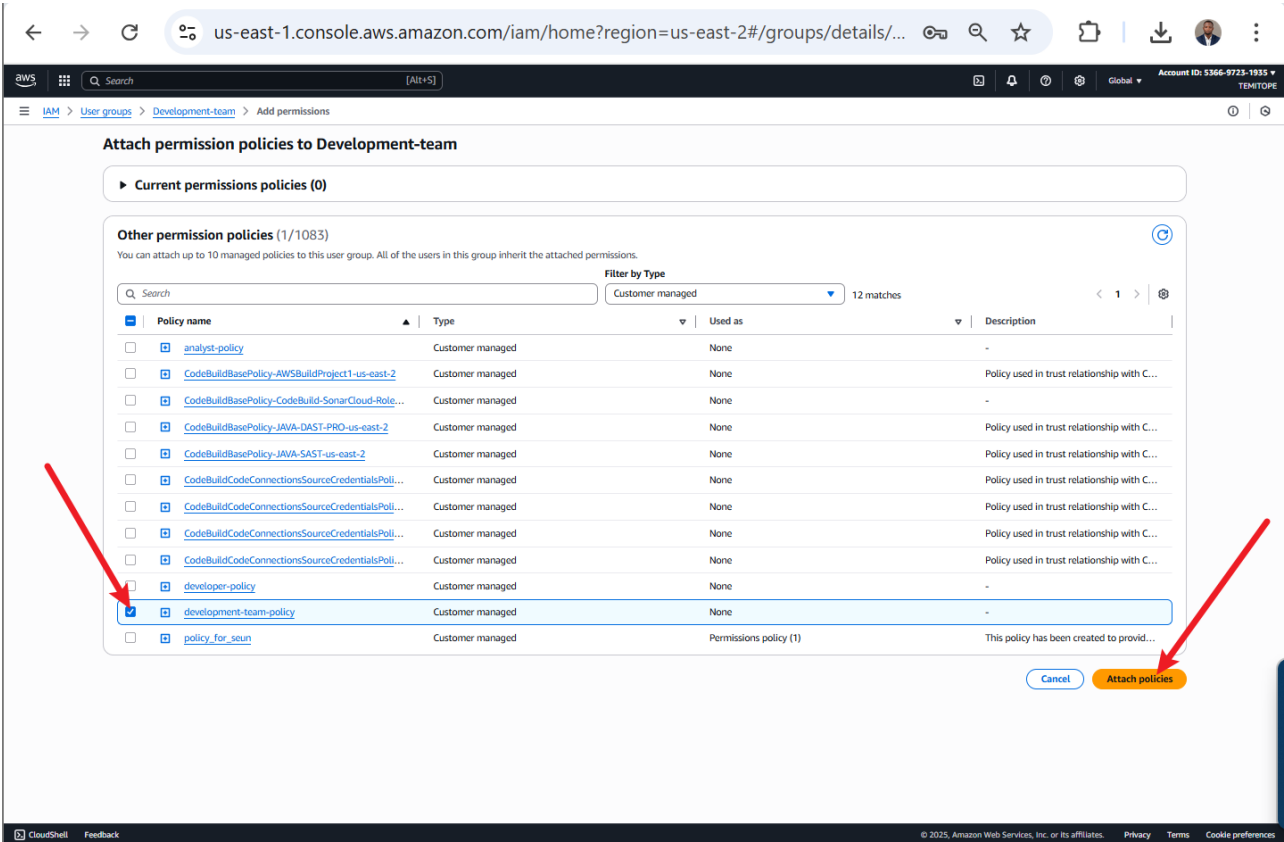
42. Click on Permission Tab, Then Add Permission Button and Attach Policy



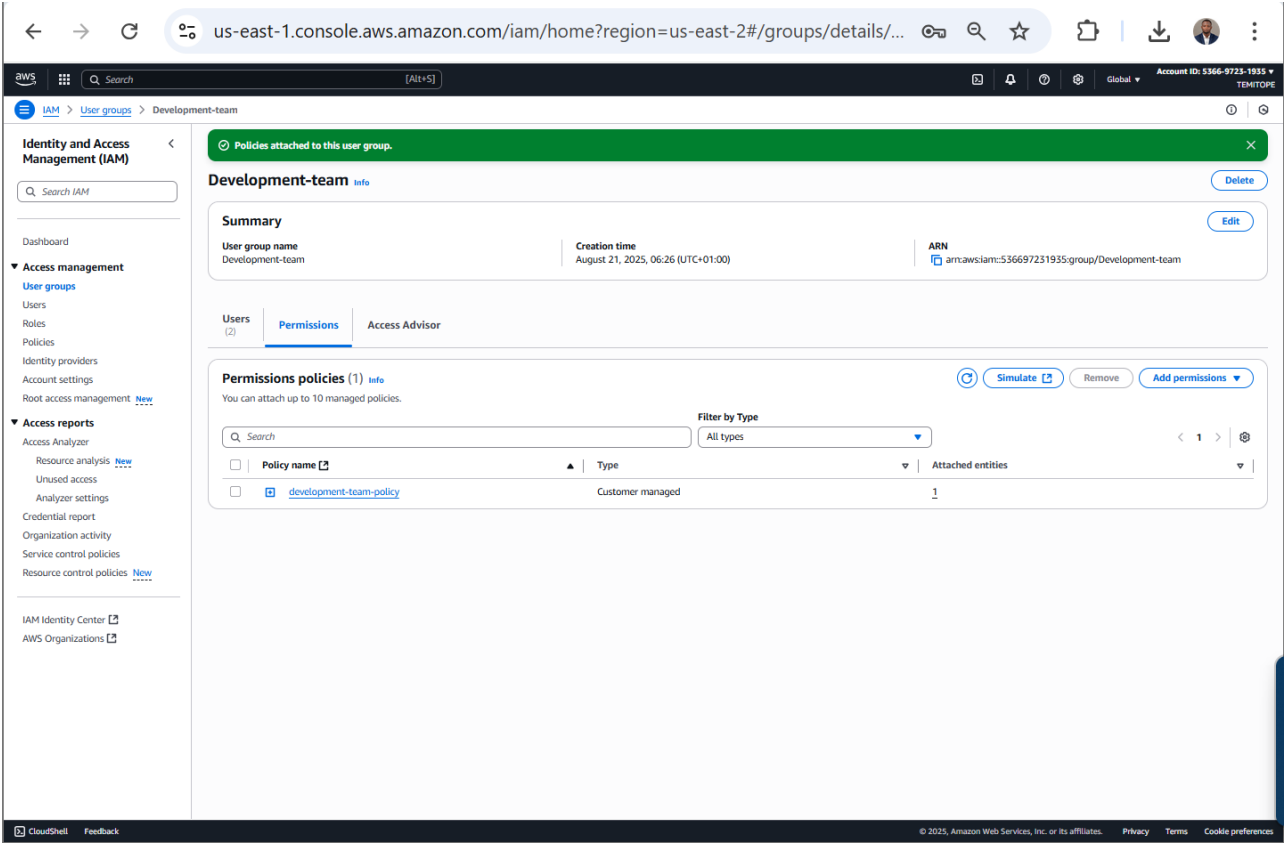
43. On Policy Name List, Filter by Customer Managed



44. Select Development-Team-Policy and Click Attach Policy

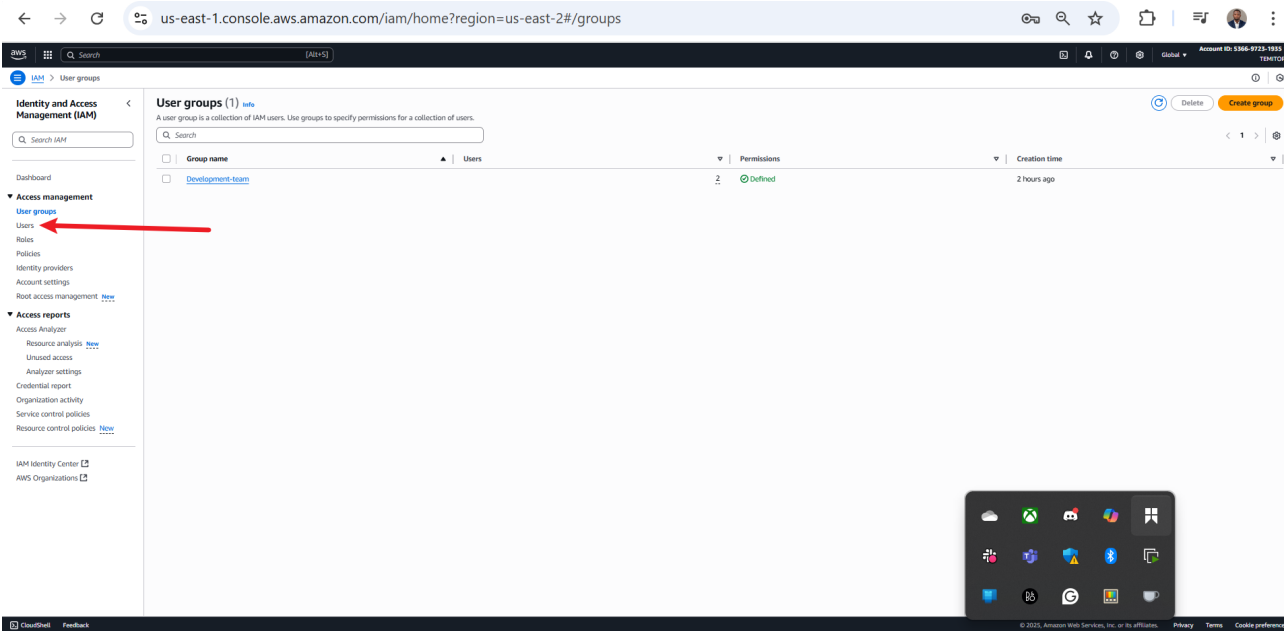


45. Policy Successfully Attached

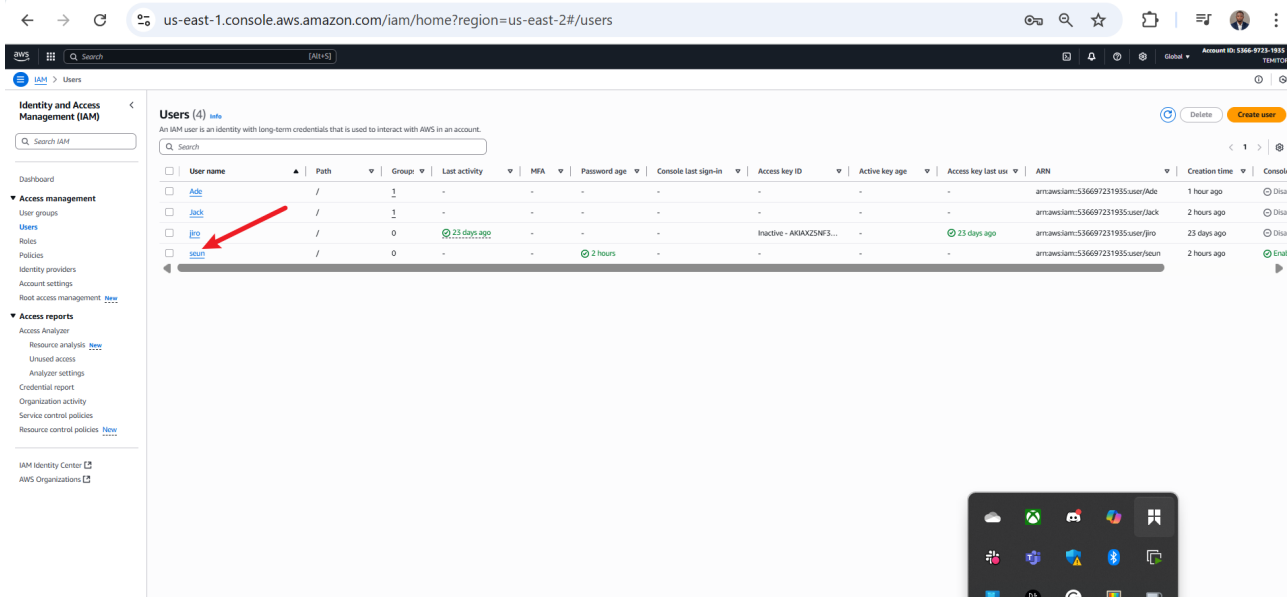


Part 3: Enable MFA for User Seun

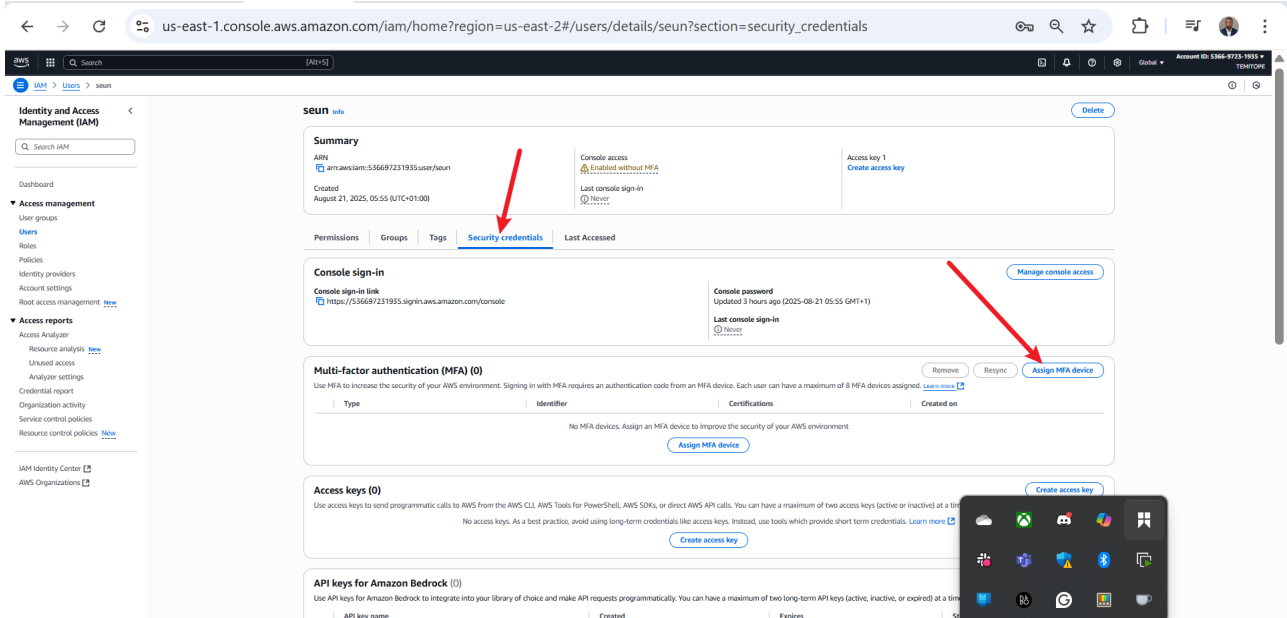
46. Click Users to Manage User



47. Click on Seun



48. Click on Security Credentials and Assign MFA



49. Enter Device Name and Choose Authentication Option

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/users/details/seun/mfa

Step 1: Select MFA device
Step 2: Set up device

Select MFA device

MFA device name
Device name
This name will be used within the identifying ARN for this device.
Seun-Auth
Maximum 64 characters. Valid characters: A-Z, a-z, 0-9, and +, =, @, _ (hyphen)

MFA device
Device options
In addition to username and password, you will use this device to authenticate into your account.

- ☐ **Passkey or security key**
Authenticate using your fingerprint, face, or screen lock. Create a passkey on this device or use another device, like a FIDO2 security key.
- ☒ **Authenticator app**
Authenticate using a code generated by an app installed on your mobile device or computer.
- ☐ **Hardware TOTP token**
Authenticate using a code generated by Hardware TOTP token or other hardware devices.

Cancel Next

50. Show QR Code, Scan QR Code, Provide Two Subsequent Codes on the Authenticator App, and Add MFA

us-east-1.console.aws.amazon.com/iam/home?region=us-east-2#/users/details/seun/mfa

Step 1: Select MFA device
Step 2: Set up device

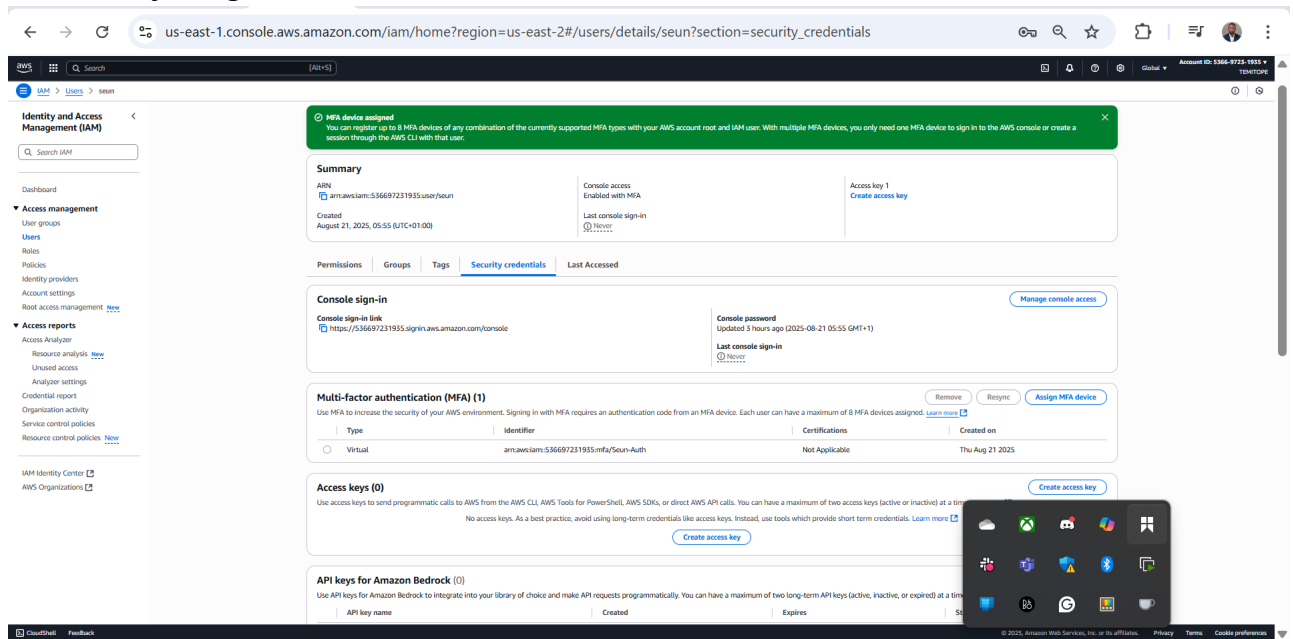
Set up device

Authenticator app
A virtual MFA device is an application running on your device that you can configure by scanning a QR code.

1. Install a compatible application such as Google Authenticator, Duo Mobile, or Authy app on your mobile device or computer.
[See a list of compatible applications](#)
2.
Open your authenticator app, choose **Show QR code** on this page, then use the app to scan the code. Alternatively, you can type a secret key. [Show secret key](#)
3. Type two consecutive MFA codes below
Enter a code from your virtual app below
153944
Wait 30 seconds, and enter a second code entry.
510972

Cancel Previous Add MFA

51. Successfully Assigned MFA



Project Reflection

Project Outcomes

The project achieved its goals:

- Understood IAM components (users, groups, policies).
- Created an EC2 policy for user Seun and an EC2/S3 policy for the Development-team group with users Jack and Ade.
- Enabled MFA for Seun, enhancing security.
- Applied best practices like least privilege, descriptive names, and MFA.

Challenges Faced

- Navigating the AWS Console was initially complex; the search bar and sidebar were key.
- Configuring policies required balancing granularity and access.
- Differentiating user vs. group permissions was initially unclear.
- Setting up MFA involved learning authenticator apps and QR code scanning.

Lessons Learned

- Granular policies enhance security.
- Groups streamline permission management for teams.
- MFA significantly boosts account security.
- Clear documentation prevents errors.
- Best practices like least privilege and MFA are practical and effective.

Potential Improvements

- Enable MFA for Jack and Ade for comprehensive security.
- Include IAM role creation (e.g., for AWS services).
- Test policies using the IAM Policy Simulator.

- Ensure consistent naming in documentation.

Conclusion

The project built practical IAM skills, including policy creation, user/group management, and MFA implementation for secure AWS resource management. Challenges like console navigation, policy setup, and MFA configuration were overcome, reinforcing best practices. Adding MFA for all users, role-based exercises, and policy testing would enhance future iterations. I'm now confident in applying IAM and MFA to real-world AWS scenarios.